M362 Unit 3
UNDERGRADUATE COMPUTING

Developing concurrent distributed systems

Interacting processes

Unit 3

This publication forms part of an Open University course M362 *Developing concurrent distributed systems*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at http://www.open.ac.uk where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit http://www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

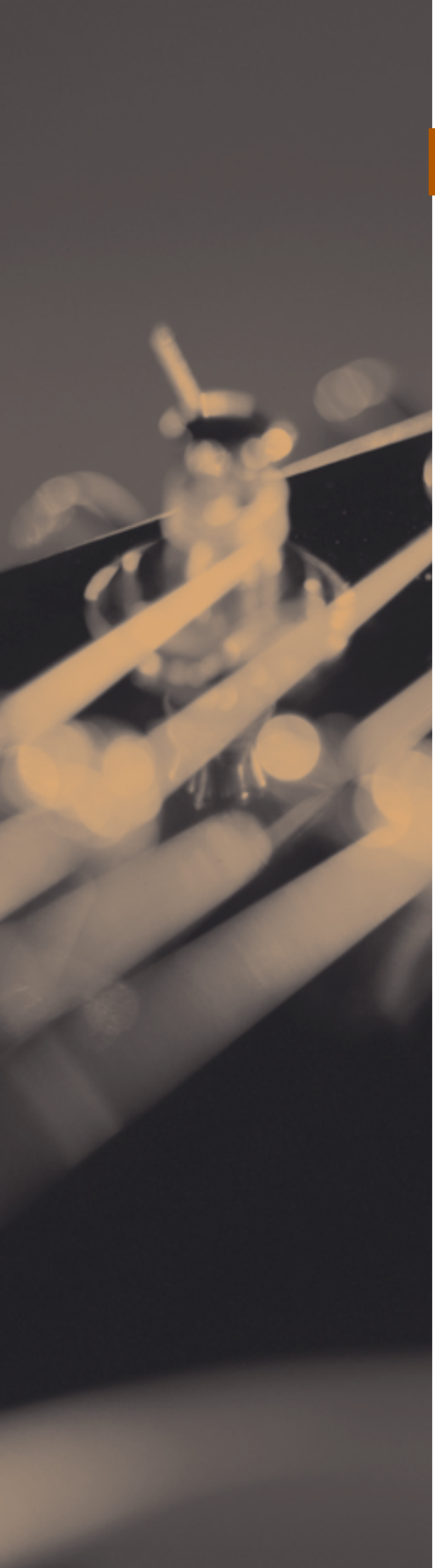
Edited and designed by The Open University.

Typeset by The Open University.

Printed and bound in the United Kingdom by Martins the Printers, Berwick-upon-Tweed.

ISBN 978 0 7492 1591 0

1.1



CONTENTS

M362 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Chair, author and academic editor

Janet van der Linden

Authors

Anton Dil

Brendan Quinn

Michel Wermelinger

Critical readers and testers

Henryk Krajinski

Barbara Segal

Mark Thomas

Richard Walker

Yijun Yu

External assessor

Aad van Moorsel, Newcastle University

Software development

Ivan Dunn, Consultant

Course management

Linda Landsberg

Carrie Lewis

Barbara Poniatowska

Julia White

Media development staff

Ian Blackham, Editor

Sarah Gamman, Contracts Executive

Jennifer Harding, Editor

Phillip Howe, Media Assistant

Martin Keeling, Media Assistant

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Sue Stavert, Technical Testing Team

Andrew Whitehead, Designer and Graphic Artist

Thanks are due to the Desktop Publishing Unit of the Faculty of Mathematics, Computing and Technology.

1 Introduction

In *Unit 2* we studied processes as the mechanism through which things get done in computer systems and, more specifically, how things are done concurrently. In this unit we study the need for processes to communicate with other processes in order to achieve their goal.

The study of communication between processes is sometimes referred to as IPC, inter-process communication. Communication can be either *direct*, taking the form of processes sending messages to each other (a topic we will study in Block 2) or *indirect*, arising from the fact that processes share data. The latter will be the focus of this unit.

When processes share data, the access to such a shared resource needs to be carefully controlled. Without such control two or more processes might attempt to access the data at exactly the same time, leading to unexpected results.

There are a number of mechanisms specifically developed to control the access to shared data, known as concurrency control mechanisms, for example critical regions, semaphores and monitors. We will explain how they work, and how they can be used to solve problems involving shared data.

The Java programming language has many features that support concurrency. In this unit we will begin our study of these features, and find out how to write concurrent programs using Java. The programmer needs to be aware of some common pitfalls, and in particular needs to be aware that unless care is taken programs may suffer from deadlock.

During the first part of the unit, we will discuss the concepts using the term *process* as the general idea or model of a program in execution. From Section 4 onwards we will be dealing with the Java implementation of processes, and use the term *thread*.

1.1 The aims of this unit

In this unit you will study:

- the idea of a shared resource and processes interacting over the use of a shared resource (Section 2);
- the development of a number of concurrency control mechanisms and how they work (Section 3);
- the Java facility to write concurrent programs (Section 4);
- the multiple readers, single writer problem – a ‘classic’ problem in the study of concurrency, which we will study how to program in Java (Section 5);
- the problem of deadlock and how to prevent it happening (Section 6).

The first part of this unit (Sections 2 and 3) will involve mainly reading and exercises. From Section 4 onwards, where we study the implementation in Java of concurrency, you will be required to do some practical programming activities.

2 Process communication and shared resources

Processes that are working together often need to access the same data or the same hardware resource, collectively referred to as **shared resources**. The use of a shared resource provides an opportunity for communication between such processes – one process may produce data and another process may use this data.

In this section we begin by looking at the idea of shared resources, as this is central to process communication. We then look at an example of process communication and discuss some of the things that can go wrong if process communication is not done in a controlled manner.

2.1 Shared data resources

As an example of a shared resource consider a container capable of holding water that has an inlet and an outlet. One process can add water through the inlet, and another process can take water out through the outlet, as shown in Figure 1.

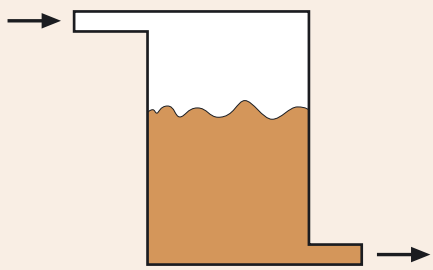


Figure 1 Water container as a shared resource

Such a model where two (or more) processes work together with a shared resource is often referred to as the **producer–consumer model**.

In real life we can see this model applied to, for example, people working in an office: one office worker may place items in an in-tray, and another worker may take items out of the tray to deal with them appropriately. Another example would be students sending in their assignments electronically to their tutor's download area, ready to be retrieved and marked by the tutor.

In general, we say that in the producer–consumer model:

- one process, the producer, writes data to the shared resource;
- another process, the consumer, reads data from the shared resource.

Buffers

An example of a shared resource in computing terms is that of a **buffer**, as introduced in *Unit 2*. When data is transferred from a user application to, for example, a printer, a producer process writes data to the buffer, and the process handling the printer takes data from the buffer. A buffer is used to smooth out irregular bursts of activity by the processes that share it.

A buffer is a linear form of storage, but can be used as if it is circular, and can then be referred to as a **circular buffer**. This is shown in Figure 2.

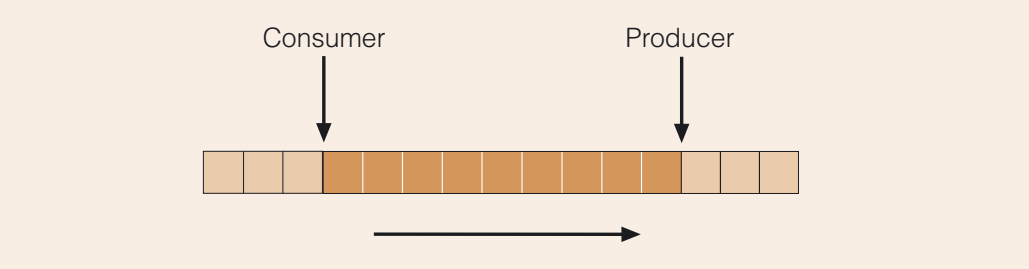


Figure 2 Sections in a circular buffer, with some slots currently unused – both processes move from left to right

In Figure 2 we see an area of memory in use as a circular buffer: on the left are the slots that have been read by the consumer process, in the middle are the slots that the producer has filled with data, but which have not yet been read, and on the right are the slots that have not yet been written to by the producer.

Even if the memory is very large, over time the number of items the producer writes to the buffer may exceed the buffer's capacity. The circular buffer makes use of the fact that data that has been read can be overwritten, and is specified as follows.

1 When either process reaches the end of the buffer, it starts again at the beginning.

2 When the producer and consumer point at the same slot in the buffer then:
   - the consumer may proceed in the case where the data has not yet been read;
   - otherwise the producer can proceed.

Therefore, when a producer has filled the buffer to its capacity, it starts again from the beginning, provided that this contains data that has now been read. This is shown in Figure 3.

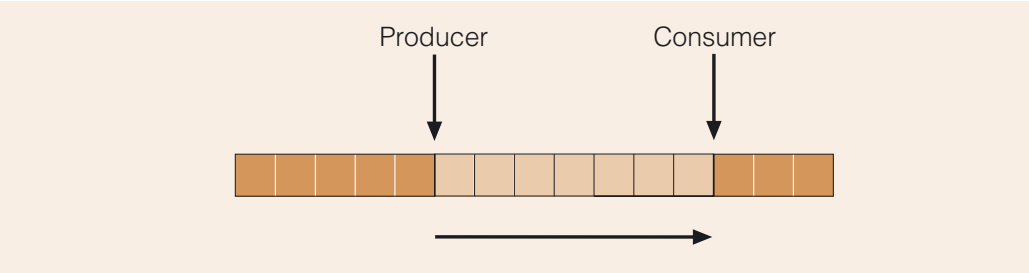


Figure 3 Sections in a circular buffer, with some slots overwritten by the producer

Problems when sharing data

The sharing of resources provides a powerful mechanism for communication between processes. However, there are a number of problems that can arise if access to the shared resource is not controlled.

- The consumer may attempt to read an item while the producer is part of the way through writing to the shared resource. This could lead to the data being corrupted in some way.
- The consumer may attempt to read from the resource before the producer has managed to produce anything.
- The producer may overwrite the shared resource before the consumer has had a chance to read it, in which case a piece of data has effectively been lost.
- A data item may end up being read several times, when it was only meant to be read once.

Therefore, there need to be protocols to ensure that when several processes share data they access it in a controlled manner in order to avoid the problem of **race conditions**. In general, race conditions exist if the final outcome of a process is dependent on the timing or sequence of execution of other processes. So, if a reading process and a writing process (or indeed more than one of each) are involved in reading and writing shared data concurrently, then the final result depends on which process does what when – hence the idea of a race between the processes.

2.2 Condition synchronisation and mutual exclusion

The buffers shown in Figures 2 and 3 contain some data but also have some empty slots. Therefore, in each scenario, the consumer process is able to consume data because the buffer is not currently empty; the producer process is able to put data into the buffer because it is not full at this point.

This shows that there are conditions that should hold for each process before it is able to continue processing. For a consumer process the condition should hold that there is some data that can be consumed, i.e. the *buffer is not empty*. For the producer the condition that the *buffer is not full* should hold. Each process must therefore make use of **condition synchronisation**. As the name suggests, condition synchronisation is a mechanism to ensure that a process is blocked if some condition is not satisfied, i.e. the process must **synchronise** with the condition for progress to be made.

We mentioned the possible problems that can arise from uncontrolled access to shared data. Access to shared data can be brought under control by allowing only one process at a time exclusive access to the shared resource. This is known as imposing **mutual exclusion**. Throughout this unit you will see a number of mechanisms for imposing mutual exclusion.

2.3 Potential problems – safety and liveness

Concurrent programs are more difficult to write correctly than sequential programs. This is because sometimes errors only become apparent when the program is running under certain conditions.

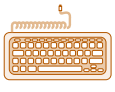
When a program is running there are two properties that need to be satisfied:

- **safety** means that nothing bad happens;
- **liveness** means that something good eventually happens.

In this subsection we will explore what we mean by each term then we will discuss some of the reasons why a concurrent program might not possess these properties.

Safety

In the context of concurrent programs, code is described as safe if it functions correctly, i.e. if it performs according to the given specification, even when executed simultaneously by multiple processes.



Activity 3.1

Race conditions in the producer–consumer problem.

If a specification is violated, the system as a whole, or the individual objects within it, may end up in an **inconsistent state**. For example, with reference to the specification of the circular buffer we gave earlier, a producer process will put the buffer in an inconsistent state if its pointer goes out of the buffer (violating part 1 of the specification), or if it proceeds when it is not allowed to (violating part 2). Similarly, a consumer process can put the buffer into an inconsistent state if it proceeds at the wrong moment, or if it reads outside the buffer.

The mutual exclusion requirement discussed above is about preserving the **consistent state** of a resource shared by multiple processes, and hence the safety of the system.

Liveness

By liveness we mean that the program must make some form of progress, and perform according to its specification. In the example of the producer–consumer problem with a buffer, there may be liveness issues if the condition synchronisation is not implemented correctly. For example, suppose that a consumer process is given access to the buffer when it is empty. If the consumer process does not then give up its hold of the buffer, the producer process is effectively refused access. As a result the producer cannot insert anything into the buffer, which means that the condition of the buffer cannot change and no progress will ever be made.

There are a number of known liveness failure problems, for example in **deadlock** two or more processes hold resources in such a way that none of the processes can make progress. In this scenario each process is blocked, waiting for a resource that is held by another process, and none of the processes will give up the resource it is holding.

Dining philosophers

The dining philosophers problem is a classic concurrency problem which demonstrates deadlock. In this problem we imagine five philosophers sitting around a table, sharing a huge bowl of spaghetti, as shown in Figure 4.

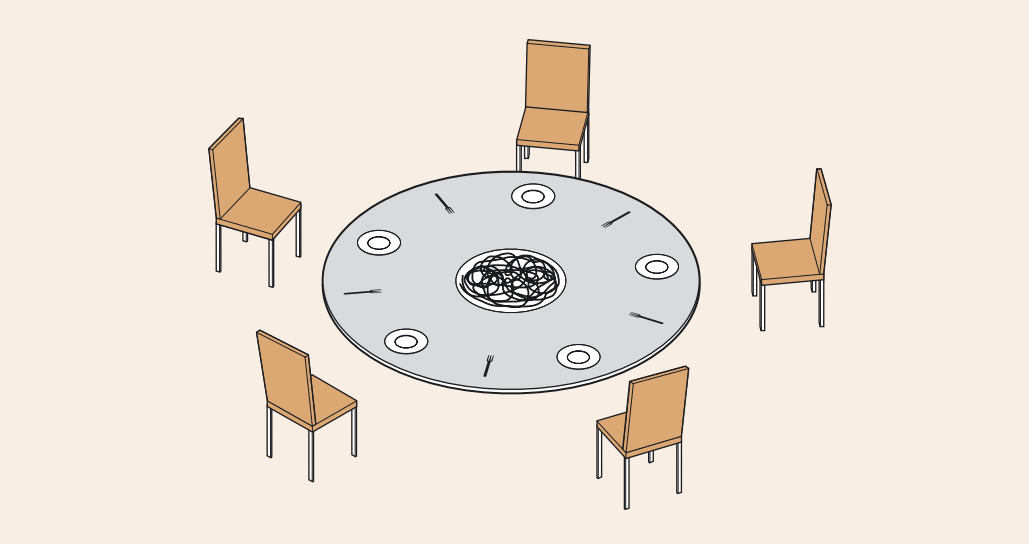


Figure 4 Dining philosophers around a table

There are five forks. Each philosopher spends his time either eating or thinking. While thinking, a philosopher does not hold any forks. When getting ready to eat, a philosopher picks up first his right fork, and then his left fork, because he requires two forks to eat from the spaghetti bowl. Note that the bowl used for the spaghetti is an interesting type of bottomless bowl, which never gets empty.

Magee and Kramer (1999) suggest that the philosophers need to share forks because they are not as well paid as computing professionals!

If all the philosophers get hungry at the same time, they would each pick up their right fork at the same time. They are then in the unfortunate situation that they are all blocked, since each philosopher is waiting for a fork held by his neighbour. The system can make no progress and is said to be in deadlock.

Livelock

Livelock is similar to deadlock, but rather than processes being blocked, and not able to do anything, this describes a situation where a process is continually executing an operation but without getting nearer to a condition becoming true. There is a sense that the system as a whole is making no progress, and is repeating itself to no effect.

An example, borrowed from real life, is where two people meet each other in a narrow corridor. Each person politely steps aside in order to let the other person through. If they both initially move to the same side of the corridor, and continue to move simultaneously, the effect may be that both people sway from side to side, without making any progress because they continue to move the same way at the same time.

Fairness and starvation

When writing programs in which several processes are competing for resources, we must ensure that the system will be fair. The **fairness** of a system is about whether each process gets enough access to (necessarily) limited resources to make reasonable progress. An example of unfairness is **starvation**, which occurs when one or more processes in a program are continuously blocked from gaining access to a resource and, as a result, cannot make progress. Clearly fairness is an issue relating to the liveness property.

Note that in our discussion a resource can be a variety of things including hardware, software and data. For example, a process may be starved if it cannot gain access to the CPU and is therefore not given a chance to execute. This would be partly determined by the resource allocation policy for the process scheduler as discussed in *Unit 2*. In the case of data, if one process keeps its hold on a data object without ever giving it up, then other processes are denied access to that object. Both these scenarios are forms of unfairness.

Dealing with concurrency

Throughout this unit we will look at different mechanisms and algorithms for dealing with concurrency. The safety and liveness properties will be central in our investigations.

In this section we have used the concept of a circular buffer, and the producer–consumer model to introduce some important ideas relating to accessing shared data. However, much of what we have discussed applies to any operations on shared data, whether it is about writing data to a database, the use of a shared file, or several threads using a shared global variable such as a Boolean flag or a counter.

We have used imaginary scenarios to illustrate some of the problems that can arise in concurrent systems: the circular buffer, dining philosophers and people bumping into each other in a corridor. These examples are chosen because they allow us to simplify some complicated points. This does not mean that the problems we are talking about are not serious. Race conditions are very important to understand; they come up in practically every setting, and as mentioned before, they may not become apparent during development, but only manifest themselves during the execution of a program. At that point it is probably too late to correct the mistake.

The northeast blackout of 2003

On 24 August 2003, a large part of the northeast of the USA and the province of Ontario, Canada was affected by a power cut. Some fifty million people were without power for up to two days.

Figure 5 Map showing the geographical spread of the 2003 northeast blackout

The immediate cause appeared to be the high summer temperatures, which had led to a high demand for power (air-conditioning systems and fans), combined with a sagging of the power lines. As lines started to sag, they ended up touching the tops of trees, thus causing a loss of power in the immediate area. Normally, a loss of power in an area is noticed by a computerised Energy Monitoring System (EMS), which sets off an alarm. On this occasion, alarms failed to go off, or were inaudible, and staff were unaware that the power blackout was spreading rapidly.

The failure of the alarm system was eventually traced back to a fault in the software of the EMS. The system consisted of approximately one million lines of code, and was written in the C and C++ programming languages. About eight weeks after the blackout, the bug was unmasked as an error due to race conditions. Two processes were competing for access to a shared data structure, and due to some error, the processes were allowed access simultaneously.

That corruption led to the alarm-event application getting into an infinite loop and 'spinning'. After the alarm function crashed, unprocessed events began to queue up, and within thirty minutes the EMS server hosting the alarm process folded under the burden. A backup server kicked in, but it also failed. By the time operators figured out what was going on and restarted the necessary systems, hours had passed, and it was too late.

What is interesting is that the system had been tested exhaustively, and had been operational for over three million hours in which nothing had ever exposed that bug before.

Source: Poulsen (2004)

The northeast blackout.

SAQ 1

(a) What are race conditions?

(b) In a circular buffer, if the data is being overwritten by a producer before it is consumed by the consumer, what problem has occurred?

(c) In deadlock, a process is blocked while waiting for a resource that is held by another process. In the dining philosophers problem, what would be the resource?

(d) What is the difference between deadlock and livelock?

ANSWER.....

(a) Race conditions are when the final outcome of a process is dependent on the timing or sequence of other processes.

(b) This is an example of a race between the consumer and the producer processes to get to the shared memory: a race condition has occurred.

(c) In the dining philosophers problem, forks are the resource.

(d) In deadlock, one or more processes are not able to execute at all. In livelock, a process is executing operations without progressing.

3 Mechanisms to ensure mutual exclusion

In this section we take a look at some well-known concurrency mechanisms that have been developed to allow multiple processes to access shared resources in a safe manner. The main issue that these mechanisms seek to address is the implementation of mutual exclusion and condition synchronisation.

First we will discuss the concepts of atomicity and critical regions, both of which underpin the software mechanisms that have been developed to control concurrency. We will also, briefly, look at the hardware support available on most computers.

Next we will discuss a software solution developed during the 1960s. Although it is now no longer in use, understanding the issues the solution needed to resolve is valuable, as is the appreciation of the solution as a historical step in the development of such concurrency mechanisms.

We will finish by looking at semaphores (which were first introduced in 1968 and are still in use today) and monitors (a concept that was first introduced in 1974 and turned out to become the model on which the Java concurrency mechanism is based).

In the 1960s and 1970s the study of concurrency was relevant to only a handful of people concerned with the design of operating systems, and was not a topic that the average software engineer had to be aware of. However, with the growth of the internet and the widespread use of Java with its inbuilt concurrency mechanisms, concurrency is increasingly becoming a useful technique when developing applications where responsiveness and throughput are important factors. We will see that some of the algorithms and mechanisms developed during the 1960s and 1970s are still important today.

3.1 Atomic actions

We begin the discussion of mutual exclusion mechanisms by defining atomic actions. An **atomic action** is an action that is indivisible – that is, either it happens completely, or it does not happen at all. A machine instruction is an example of an atomic action, as this is the fundamental instruction that the CPU works with during the fetch–execute cycle. The CPU checks for interrupts at the end of each cycle, not during the cycle. Hence a machine instruction is by definition indivisible.

The word atom is derived from the Greek word *atomos*, meaning indivisible. It was originally applied in science as being the smallest part of a chemical element – although we now know that atoms can be split after all!

An atomic action is defined by the following criteria:

1 a process switch cannot happen during an atomic action so,

2 no other operation can be interleaved with an atomic action, and

3 no other process can interfere with the manipulation of data used by an atomic action during the time the atomic action is taking place.

We can also define atomic actions in terms of crash resilience: that is, an atomic action is 'all-or-nothing'. If a crash occurs, either the results are permanently stored, or it is as if it never happened. This will be discussed in *Unit 6*.

The notion of atomic actions is further developed by Andrews, when he talks about **fine-grained** and **coarse-grained actions**:

> An atomic action is a sequence of one or more statements that appear to be indivisible; i.e. no other process can see an intermediate state. A fine-grained atomic action is one that can be implemented directly by an indivisible machine instruction. A coarse-grained atomic action is a sequence of fine-grained actions that appears to be indivisible.
>
> Andrews (1991)

Atomic actions are a useful concept in the study of concurrency. To see why, consider the following example.

In a web application that supports customers buying items online, it is necessary to keep an up-to-date record of the stock levels for each item that is for sale. If a customer wants to buy an item, the system can check the stock level and update it with the new value. A code statement such as the following would be executed each time a customer wants to purchase an item:

```
if stockLevel > 0
then stockLevel = stockLevel – 1
```

This statement consists of multiple machine instructions: *read*, *test* and *write*.

If several customers want to buy the same item at the same time, then the danger exists that the statement is invoked simultaneously by a number of processes.

Table 1 below shows a potential scenario for two processes, process *A* and process *B*, both attempting to execute the statement on a uniprocessor system. We assume that at the start of this scenario the value for *stockLevel* is 3. After processes *A* and *B* have run, *stockLevel* should have the value 1, but let us see what might happen instead.

Table 1 Two processes reading *stockLevel*

Step	Process *A*	Value of *stockLevel* in register *A*	Process *B*	Value of *stockLevel* in register *B*
1	Read *stockLevel* into register	3		
2	Process *A* times out; its state is saved	3		
3		3	Read *stockLevel* into register	3
4		3	Test *stockLevel*; value found to be 3	3
5		3	Deduct 1 and write *stockLevel=2*	2
6	Process *A* resumes; its state is reinstated	3		
7	Test *stockLevel*; value found to be 3	3		
8	Deduct 1 and write *stockLevel=2*	2		

The columns in the table show when each part of the statement is executed by each process, and what the value is for the *stockLevel* variable held in the register for that process. So, on the first line, in step 1, we see that process *A* has been selected by the scheduler to run, and process *A* executes the *read* part of the statement. The value that is being read, here 3, is stored in the register for process *A*. The scheduler

uses pre-emptive scheduling, and after a certain amount of time, process *A* is called to a halt (step 2) and process *B* is given a chance to run (step 3). Process *B* manages to complete the entire statement, and process *A* is scheduled again and now also completes the statement.

Table 1 shows that the *write* operation made by process *B* in step 5 has been lost and is overwritten by the update by process *A*. This is referred to as the **lost update problem**, which occurs because uncontrolled access to a shared resource has been allowed. The lost update problem is an example of the existence of race conditions. The focus of our study in this unit will be the avoidance of race conditions.

The example shows that what looks like a simple statement, or at least a statement which forms one logical operation, is actually broken down into several smaller steps, *reading the value, testing the value* and then *decrementing and writing the value*. It is possible for the operating system to cause an interrupt between these smaller steps. The problem of the lost update would have been avoided if the statement as a whole had been treated as an atomic action.

The concepts of fine-grained and coarse-grained atomic actions can be viewed as the two extremes of a sliding scale. At one extreme is the indivisible machine instruction, where a process can read or write a single byte of memory without fear of interference from any other process because such a read or write is made indivisible by the hardware. At the other extreme of the scale an entire program could be seen as a coarse-grained atomic action. At this latter extreme no interleaving and concurrency would be allowed and thus we would, in essence, be dealing with a sequential program. In this course we will mostly be dealing with coarse-grained atomic actions which fall somewhere in between the two extremes.

There are a number of possible ways to write such coarse-grained atomic actions, as we will see in the remainder of this section. In the next section of this unit (Section 4) we will go on to discuss the Java mechanism for implementing atomic actions.

Exercise 1

An interactive game application has a graphical user interface (GUI) to display the graphics of the game. Several players can play against each other, and the aim of the game is to move objects so that they end up in each player's area. The players can select an object that needs moving (several players can try to move the same object at the same time), and the game application itself is also capable of moving objects about in a random way for some added complexity.

As part of this application we have a method `moveBy`, to change the position of an object, defined as follows:

```
private void moveBy(int xChange,int yChange)
{
  xCoor = xCoor + xChange;
  yCoor = yCoor + yChange;
}
```

(a) Do you think the method `moveBy` should be made an atomic action and, if so, why?

(b) If `moveBy` were an atomic action, what type of atomic action (fine-grained or coarse-grained) would it be?

(c) Is there a link between the concept of atomic actions and the property of safety?

Discussion .....

(a) Yes, `moveBy` should be made atomic because it updates the state of an object and this update should be indivisible. If the process is not atomic, when one player process is in the middle of moving an object, another player process could potentially begin to move the same object. The changes to the x- and y-coordinates should always be carried out in one single action, otherwise the graphical object might be left in an unintended or invalid state.

(b) The method `moveBy` would be a coarse-grained atomic action because it involves more than one machine instruction.

(c) Yes. The safety property says that nothing 'bad' should happen to an object. Clearly, that is exactly what the concept of atomic actions is about. In the example of the `moveBy` method, it may be bad for an object to have its x-coordinate changed, but not its y-coordinate. By not treating `moveBy` as atomic, the object might end up in a position where it was never intended to be.

3.2 Defining a mutual exclusion protocol

Remember that mutual exclusion means that only one process can access a shared resource at a time. One approach to achieving mutual exclusion is through **protocols**, i.e. agreed procedures that the concurrent processes will adhere to.

We begin the definition of such a **mutual exclusion protocol** with the notion of a critical region. The source code of a program can be divided into critical regions and non-critical regions, as shown in Figure 6.

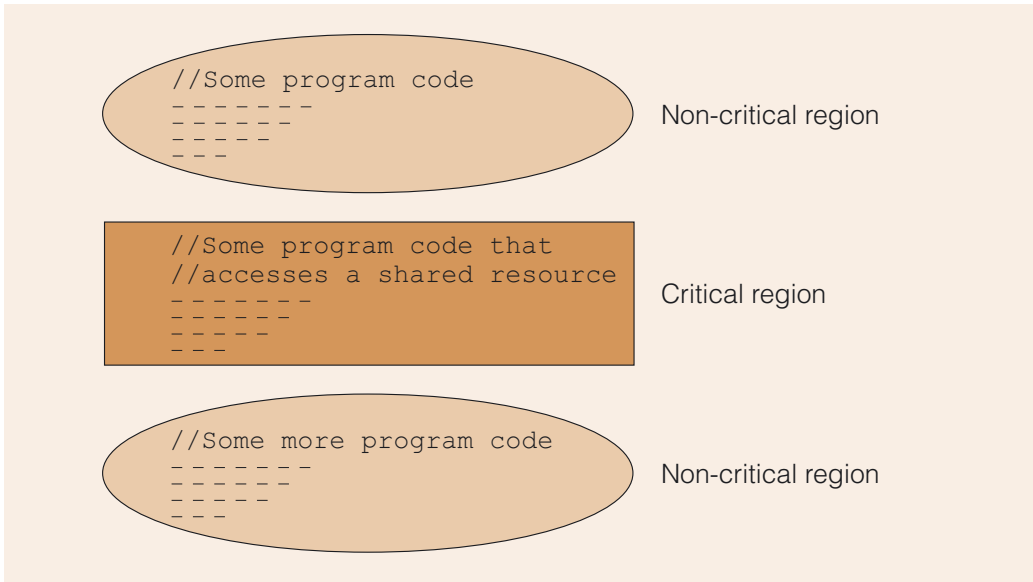


Figure 6 Critical regions and non-critical regions in the source code of a program

The **critical region** is the section of code in which a shared resource is being accessed. The critical regions for a number of processes may contain the same code, or the code may differ from process to process. For example, a number of processes might be trying to read from a circular buffer, all using the same code, or one process may be executing code to write to the buffer, while another is executing code to read from it.

Critical regions in a program need particular protection when processes are executing concurrently. To avoid race conditions, a critical region must be executed under mutual

exclusion. On the other hand, non-critical code can safely be executed by processes executing in parallel.

The critical region would not normally consist of just a single machine instruction, but will typically be made up of two or more such fine-grained atomic actions. Processes obeying a mutual exclusion protocol are required to notify other processes that they are entering or leaving such critical regions, by using what are called **entry protocols** and **exit protocols**, before and after the critical region, as shown in Figure 7.

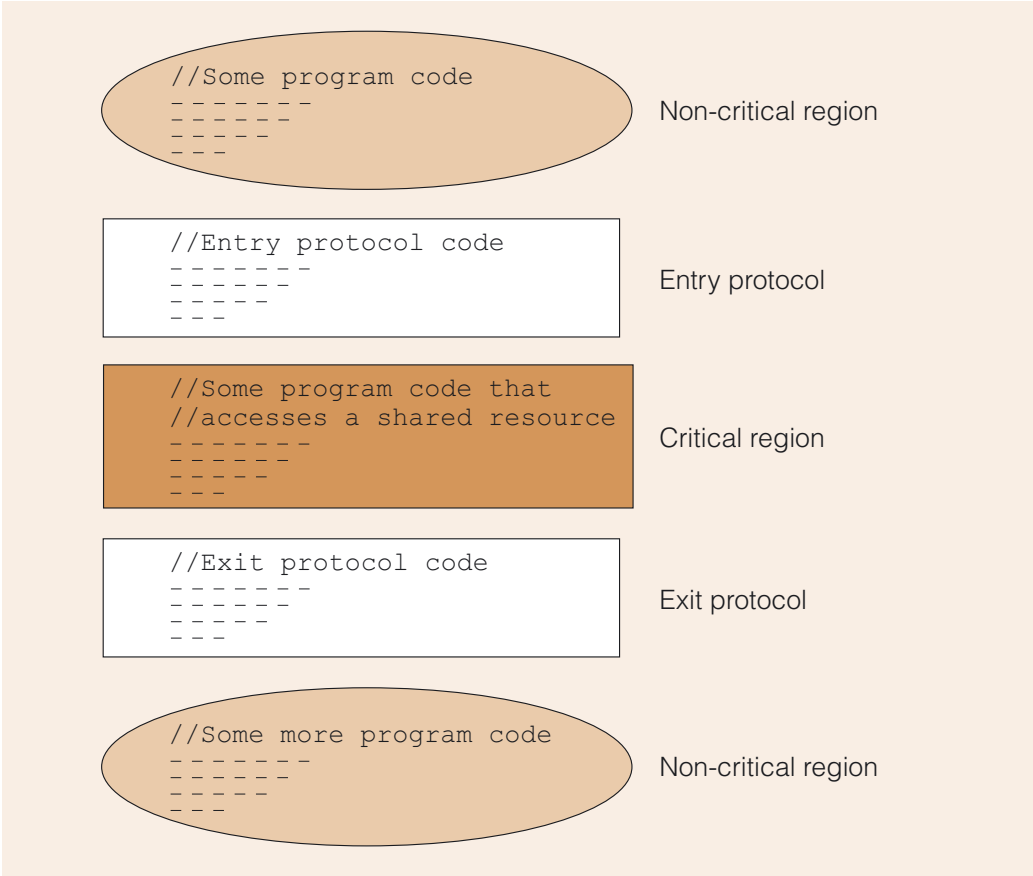


Figure 7 Critical region surrounded by entry and exit protocols

The entry protocol is the code that must be executed by a process prior to entering a critical region, and is designed to prevent the process from entering the critical region if another process is already in a critical region that accesses the same data resource. An exit protocol is the code that the process must execute immediately on completion of its critical region. This signals that the resource is now available to other processes. Together, the entry and exit protocols are designed to ensure mutual exclusion.

Here we emphasise again that the fact that two or more processes have critical regions does not mean that their executions should never overlap. What it means is that *the execution of their critical regions* should not overlap.

Design issues for the entry and exit protocols

So, how do we implement entry and exit protocols? It is important that the entry and exit protocols are themselves atomic actions, or we might just shift the problem from one place to another.

Imagine what would happen if we simply introduce a Boolean variable, `busy`, that processes must check before entering their critical regions. The action of testing whether such a variable is *`false`*, and then setting it to *`true`* to prevent others from

entering their critical regions, may consist of more than one machine instruction. For example, process *A* could read the Boolean, find the region free, set the Boolean `busy` to `true` using a write instruction, and proceed into its critical region. Meanwhile, process *B* may have read the value of the Boolean variable in between process *A* reading and writing, and also enters its critical region. So simply introducing a Boolean variable without any further support is not sufficient, because mutual exclusion can easily fail.

We stated earlier that a concurrent program must satisfy the two properties of safety and liveness. By having entry and exit protocols, the intention is to enforce mutual exclusion, and hence satisfy the safety property. To satisfy the liveness property, the protocols need to ensure that a process will eventually enter its critical region, and that if a process is required to enter this region, it is not prevented from doing so.

The entry and exit protocols must also be implemented in such a way that we avoid deadlock and starvation. An example of deadlock would be if two or more processes attempted to enter their critical regions, but became stuck in their entry protocols. Starvation occurs when a process waits indefinitely to enter its critical region.

We now discuss a number of ways in which the entry and exit protocols can be implemented as atomic actions.

3.3 Hardware support for mutual exclusion

Here we discuss two ways in which hardware can assist in controlling access to shared resources such that mutual exclusion is enforced: the so-called TAS instruction and the disabling of interrupts.

Test-and-set

Similarly, the compare-and-swap (CAS) instruction is an atomic instruction that can compare the contents of a given memory location to a specified value, and if they are the same, modifies the memory location to a given new value. Again, all this takes place in one go, without interruption.

Most computers have a special instruction, called a **test-and-set** (**TAS**) instruction, which is capable of testing the value of a Boolean variable, and then setting this variable to another value. That is, a TAS instruction is a machine instruction which by definition is executed by the hardware in one go, and cannot be interrupted by the process scheduler.

This is a very helpful primitive when dealing with concurrency, as the difficulties that can arise when several processes try to read and change the value of a Boolean variable have now been removed. The TAS instruction could thus be employed in an entry protocol.

Modern computers are routinely equipped with processors which support the special instructions, such as TAS instructions, that enable mutual exclusion. However, during the 1970s and 1980s it was uncommon for computers to be equipped with hardware support for mutual exclusion.

Disable interrupts

Another way of making actions atomic is to disable the interrupts for the duration of the atomic action. This would mean that as an action starts executing, it cannot be interrupted (pre-empted) by the scheduler, and will complete its run.

However, **interrupt disabling** is not considered good practice for user programs. Event interrupts are specifically aimed at making it possible for the system code to interrupt application code whenever important tasks need doing (this may be some updating within the operating system itself), or to alert the application that a task that was waited for has now finished. It would thus be unwise to give the application software control over the running of the operating system.

Furthermore, this technique would only work on a single processor machine. If a process on a multiprocessor system had disabled interrupts, a different process executing on one of the other processors could still access the resource we were trying to protect.

SAQ 2

If a shared Boolean variable, such as *busy*, were used within entry and exit protocols, would both protocols require a TAS instruction?

ANSWER.....

The entry protocol consists of at least two instructions, checking the variable, and then setting its value, so this would require a TAS instruction to work properly as an atomic action. On the other hand, the exit protocol consists of resetting the value of the Boolean variable (without the need to check it first), so this does not necessarily require a TAS instruction.

3.4 Software implementation of entry and exit protocols

As we mentioned previously, during the 1970s and 1980s it was uncommon for computers to be equipped with hardware support for mutual exclusion. This meant that a great deal of effort was directed towards the development of solutions based on software alone.

In this section we look at a very simple algorithm for a solution, proposed by Dijkstra (1965), that turns out to be unsuitable as it stands and requires further improvements to work properly. By studying this example you will get a better feel for the difficulties of writing algorithms that must take into account all the problems posed by concurrent execution.

Attempt at a two-process solution

This algorithm is developed for two processes, named P_0 and P_1, and we will discuss the solution using Java notation. The algorithm uses a single variable, *turn*, of type `int`. The idea is that if *turn == 1* then P_1 is allowed to enter its critical region, and if *turn == 0* then P_0 is allowed to enter its critical region.

Initially, *turn* is set to either *0* or *1* (it does not matter which). The entry and exit protocols for P_0 and P_1, using this scheme, are shown in Table 2. The method *noOp* does nothing: it is simply a device to keep the process **busy-waiting** until its turn to enter the critical region arrives.

This device is usually referred to as a busy-waiting loop.

Table 2 The entry and exit protocols for a two-process system

Process P_0	Process P_1
`// non-critical code`	`// non-critical code`

`// entry protocol for P0`
`while (turn != 0 )`
`{`
`  noOp();`
`}`
	`// entry protocol for P1`
	`while (turn != 1 )`
	`{`
	`  noOp();`
	`}`

`// critical region`	`// critical region`
`// exit protocol for P0` `turn = 1`	`// exit protocol for P1` `turn = 0`
`// non-critical code`	`// non-critical code`

According to the entry protocols, process P_0 will loop while the value of `turn` is not equal to `0`, whereas process P_1 will loop while the value of `turn` is not equal to `1`. Depending upon the current value of `turn`, either the `while` loop invokes the method `noOp` continually, thereby blocking the process, or the `while` loop is not repeated, permitting the process to start the execution of its critical region.

Suppose that `turn` has been initialised to `0` and that both processes want to enter their critical regions. This means that P_0 will be able to enter its critical region but P_1 will continue to be blocked by its busy-waiting loop.

Once it has completed executing its critical region, P_0 sets `turn` to `1`, which is a signal to P_1 that it can now enter its critical region. That is, the condition `turn != 1` in P_1's busy-waiting loop is no longer `true` so the looping stops and P_1 moves to the next instruction, which is, of course, the start of its critical region.

This algorithm ensures that only one of the two processes will be in its critical region at any one time. Unfortunately, it has one major flaw: the processes are forced to take *alternate* turns to enter their critical regions. Put differently, they are in effect held in locked-step mode, and a strict sequence of $P_0 - P_1 - P_0 - P_1 - P_0$ etc. is imposed on the execution of the processes. This means that if the process whose turn is next does not want to enter its critical region (because it is still executing non-critical code), it will prevent the other process from continuing because `turn` is changed only after a process has completed executing its critical region.

Therefore this scheme can lead to starvation as it fails to ensure that a process that wishes to enter its critical region is guaranteed to do so within a reasonable time. Indeed a process can be unnecessarily prevented from entering its critical region and may be blocked indefinitely.

Software solutions to the two-process solution.

The above algorithm was further developed and improved – if you wish you can read more on this subject on the course website.

Although the preceding discussion may appear largely of historical interest it is still important to understand the pitfalls that are possible when dealing with concurrency. Concurrent programs are very difficult to write correctly, and are also hard to test – it might be that problems become apparent only under certain live conditions.

Overhead of spin locks

The busy-waiting loop used in this algorithm is also referred to as **spin lock**: a **lock** is associated with a data resource, so that only one process at a time may use the locked resource. Other processes waiting to use the resource must 'spin', i.e. check repeatedly until the lock becomes available for use. CPU time is wasted because the scheduler will continue to allocate time to a process that is busy-waiting, even while another process is in its critical region. If the critical regions are large relative to the rest of the programs they are part of, or if there are a large number of processes competing for access to the shared data, this will slow down all the processes. For example, with 10 processes competing to access their critical regions, in the worst case we could end up wasting 90% (or more) of the CPU time.

SAQ 3

The two-process software solution in Table 2 is deficient and does not satisfy all the properties for correct concurrent programs. Explain which property in particular is not satisfied.

ANSWER.....

The liveness property is not satisfied, because one process may end up having to wait indefinitely and unnecessarily.

3.5 Semaphores

In everyday life, semaphores provide a system of signals in order to enable communication visually, usually with flags, lights, or some other mechanism. In computing, semaphores are conceptually much simpler and more analogous to the signals used on railway lines to indicate to drivers whether it is safe to continue, or whether they should wait before proceeding (into a tunnel for example).

In software, a **semaphore** is a data structure that is useful for solving a variety of synchronisation problems. Dijkstra (1968) introduced the idea of semaphores as part of the operating system in order to synchronise processes with each other and with hardware. Since then semaphores have become a very important mechanism for the synchronisation of processes generally, not just for operating systems.

One way of explaining semaphores is to think of them as a 'permit'. Only one process at a time can hold the permit, and a process must get hold of the permit before it can enter the critical region. Once it has finished the critical region, it gives back the permit.

Semaphore structure: data and operations

A semaphore is a data structure, consisting of two pieces of data: an integer value, say *sem*, and a queue. The *sem* data field has to be assigned an initial value, representing the availability of some resource. Once the semaphore has been created, the only way the value can be altered is through two operations, *semWait* and *semSignal*, specified as follows:

- *semWait*: *if sem > 0 then sem = sem − 1*, else make the process that called *semWait* wait by placing it in the queue.
- *semSignal*: if there is a process waiting on the queue for this semaphore, then allow it to resume processing, else *sem = sem + 1*.

In a moment we will work through an example to understand how these operations work. Each of the operations `semWait` and `semSignal` is implemented as an atomic action. Note that in the original paper (Dijkstra, 1968) the operations were called *P* and *V*, and that they are also often referred to as `Wait` and `Signal`. Here we refer to them as `semWait` and `semSignal` in particular to distinguish them from the Java methods that we will meet later.

P and *V* were abbreviations of 'ProLagen' and 'Verhogen'. 'Verhogen' is Dutch for 'to increase', whereas 'ProLagen' is a nonsense word, made up by Dijkstra, intended to mean something like 'probeer te verlagen', i.e. 'attempt to decrease', and hence tries to address the fact that the word for 'decrease' in Dutch, verlagen, also starts with a 'V'.

When a process is made to wait as result of calling `semWait`, it becomes blocked, and gets added to the back of the semaphore's queue. The process in the queue now waits for an event to occur, i.e. a call of the `semSignal` operation by another process using the same semaphore. When `semSignal` is executed, the process that is at the front of the queue is taken out of its blocked state and starts running.

A semaphore provides a versatile mechanism and can be put to different uses:

- for mutual exclusive access to a *single* shared resource, in which case the semaphore is called a **binary semaphore**;
- to protect access to *multiple* instances of a resource (a **counting semaphore**);
- to synchronise two processes (a **blocking semaphore**).

It is important to understand that the specification of the `semSignal` and `semWait` operations remains the same, irrespective of the use the semaphore is put to. The versatility of the semaphore mechanism is achieved through correct initialisation and through the processes calling `semSignal` or `semWait` at the right points. We will discuss how this is done in the following subsections.

Using a semaphore for mutual exclusion

Here we explain how to use a binary semaphore for mutual exclusion.

If we have several processes that are sharing a resource that should only be accessed under mutual exclusion, they would be made to share one semaphore, say `sem`. The value of semaphore `sem` should be initialised to `1`.

Each process typically executes code that consists of a combination of critical and non-critical regions. Before entering the critical region, the process uses the `semWait` operation as the entry protocol and the `semSignal` operation as the exit protocol:

```
// non-critical region
...
// entry protocol
sem.semWait();
// critical region
...
...
// exit protocol
sem.semSignal();
// non-critical region
...
```

Consider a possible scenario of three processes *A*, *B* and *C* executing concurrently, using a semaphore `s` to ensure that access to their shared resource is under mutual exclusion (Figure 8).

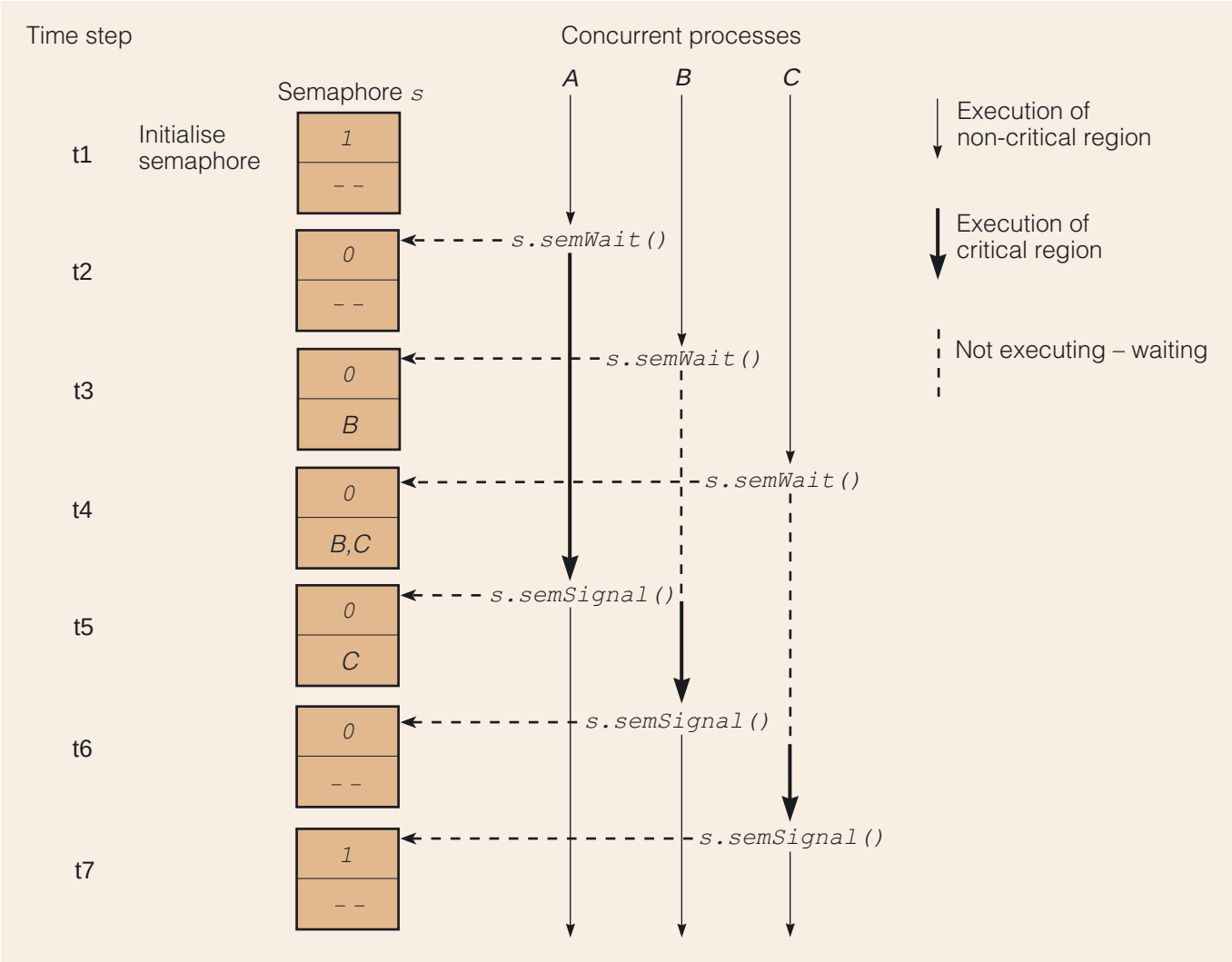


Figure 8 The execution of three processes using a semaphore for mutual exclusion

Here the semaphore is represented as a divided box where the top half displays the current integer value of the semaphore, and the bottom half displays the queue part of the semaphore, listing the IDs of the processes currently in this queue. The semaphore `s` is initialised with the value `1`, and initially no processes are waiting on this semaphore's queue. All three processes start off executing their non-critical region.

Process *A* is the first to execute the entry protocol, `semWait`, for the semaphore. At that point the value of the semaphore is `1`, so process *A* is able to progress to its critical region having set the value of the semaphore to `0`. Now process *B* comes to the point in its execution where it needs to enter its critical region. Process *B* executes the entry protocol, finds that the value is `0`, and is made to wait. Now process *C* executes the entry protocol, and like *B*, it finds that it is not allowed to enter the critical region but is made to wait for the semaphore instead.

When process *A* has finished with its critical region, it executes the exit protocol, `semSignal`. This has the effect of signalling to process *B* (which is the process at the front of the queue) that it can come out of the blocked state, and hence enter its critical region. Once process *B* has finished its critical region and executed `semSignal`, process *C* is allowed to proceed with its critical region. When process *C* completes its critical region, it executes the exit protocol and, since there are now no processes waiting for the semaphore, the value of the semaphore is increased to `1`.

Counting semaphores

As already mentioned, semaphores can also be used where there are multiple resources of the same type, all equally acceptable to a process requiring one instance of the resource. In that case the semaphore would initially be set to the number of resources available. This could be useful, for example, in the scenario where we have two printers available for processes to use. This means that up to two processes are allowed to use a printer simultaneously, but when a third process comes along it will have to wait.

To manage the access to these printers, we would initialise a semaphore with the value of the available number of resources, that is *2*. When a process requests an instance of the resource, the process is required to call the *semWait* operation, and when it finishes using the instance, it calls *semSignal*. So if two processes attempt to use the printer, each would call the *semWait* operation, and the semaphore's value would decrease from *2* to *1* to *0*. If, at that point, a third process requests an instance of the resource, that process would be made to wait.

Exercise 2

In a style similar to Figure 8, Figure 9 shows a possible scenario of several processes using a semaphore that controls access to multiple instances of a resource. There are four processes, and two instances of the resource. At step t1, the semaphore is initialised to *2* and there are no processes waiting.

Complete the diagram by indicating for each step what the value of the semaphore is, and which, if any, processes are waiting for the semaphore.

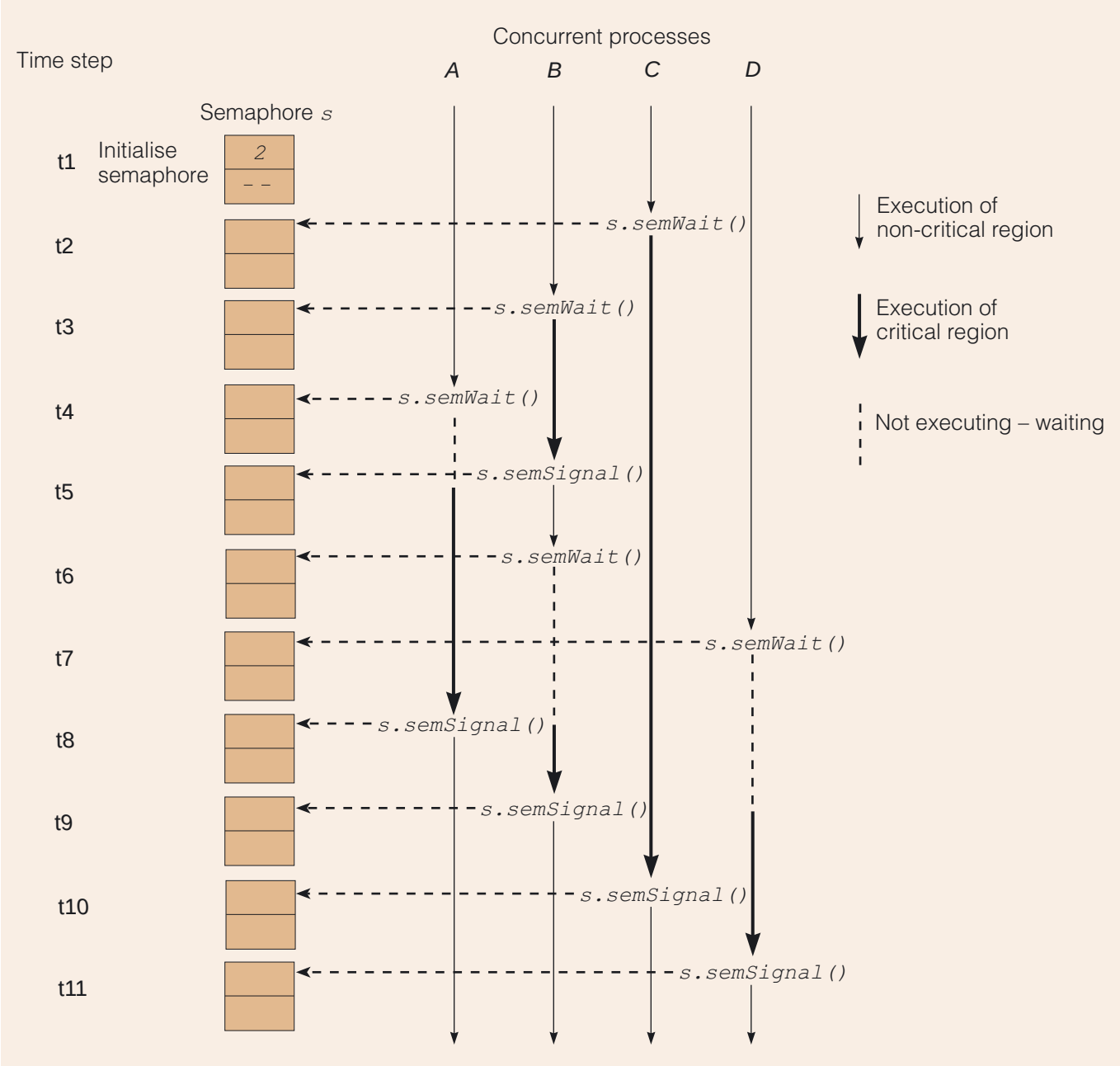


Figure 9 Incomplete figure showing the execution of four processes using a counting semaphore for mutual exclusion

Discussion .....

The diagram can be completed using the values shown below.

Time	Semaphore value	Processes waiting on queue of semaphore
1	*2*	none
2	*1*	none
3	*0*	none
4	*0*	*A*
5	*0*	none
6	*0*	*B*
7	*0*	*B, D*
8	*0*	*D*
9	*0*	none
10	*1*	none
11	*2*	none

Using semaphores for process synchronisation

In Section 2 we mentioned that in certain situations processes need to synchronise with each other, and that the synchronisation is governed by whether a condition has been satisfied or not. Such condition synchronisation can also be implemented using what are termed blocking semaphores.

As an example, consider a *consumer* process that needs to read some data from a file that a *producer* process is supposed to write to. The point in the program code at which the consumer process needs to read the input from the file is called the synchronisation point. If the consumer process reaches its synchronisation point *before* data has been written to the file, it needs to know that it should wait. Obviously, if the producer has already written data to the file, then the consumer need not wait and can pick up the data and continue processing.

In such a scenario a semaphore could be used, but in this situation it is initialised to `0`. A consumer process calls `semWait` when it reaches the synchronisation point in its code, whereas a producer process calls the `semSignal` method on reaching the synchronisation point in its program code. The pseudocode for two such programs is shown in Table 3 below.

Table 3 Pseudocode for two programs using a semaphore to synchronise

Pseudocode for *consumer* program	Pseudocode for *producer* program

`// ordinary statements`
`...`
`// section of code that requires`
`// input from file`
`sem.semWait();`
`// open file for reading`
`// read data from file`
`...`
	`// ordinary statements`
	`...`
	`// section to write data to file`
	`...`
	`// close file`
	`sem.semSignal();`
	`// ordinary statements`
	`...`

Table 3 shows that the consumer program calls the *semWait* operation before it starts using the file. The producer program on the other hand calls the *semSignal* operation when it has completed its section of code in which data gets written to the file.

Of course, the processes that execute such code will reach their synchronisation points at different times. The consumer process may reach its synchronisation point before the producer, or vice versa. These two scenarios are illustrated in Figure 10, which also shows the state of the semaphore.

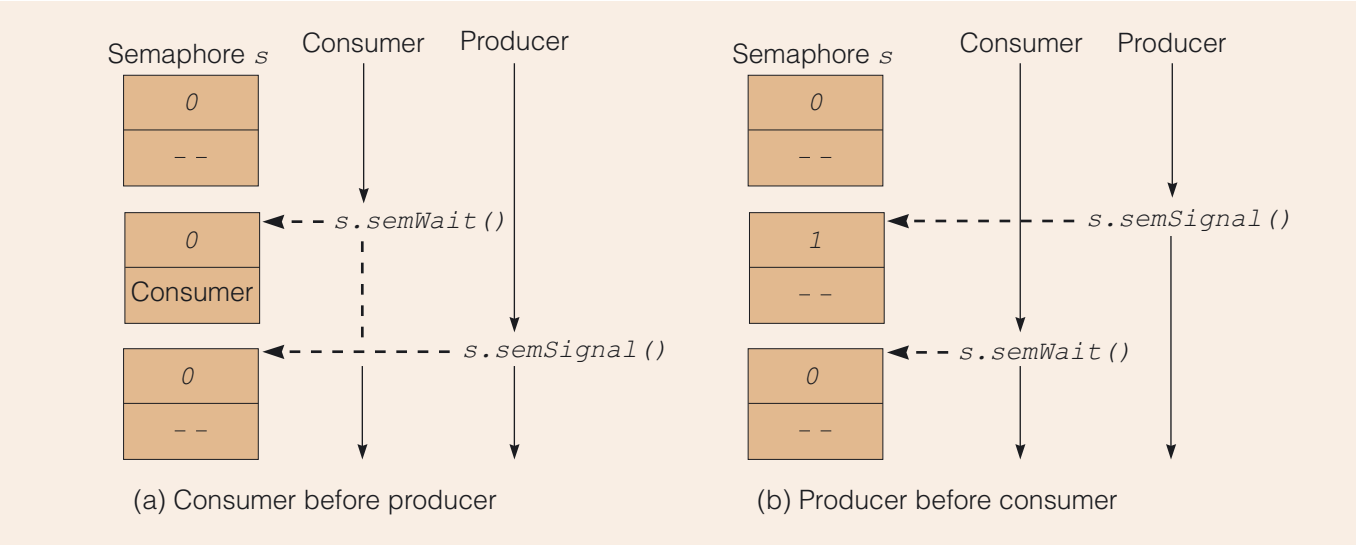


Figure 10 Using a semaphore to synchronise processes

Figure 10 (a) shows the scenario where the *consumer* reaches the synchronisation point before the producer. By calling the *semWait* operation, the consumer process is made to wait on the queue of the semaphore. As soon as the producer reaches the point where it has written data to the file, it calls the *semSignal* operation, which has the effect of taking the consumer process out of the blocked state.

Figure 10 (b) shows the scenario where the *producer* reaches the synchronisation point before the consumer. This works equally well with the way the operations *semWait* and *semSignal* have been defined. In this case, the value of the semaphore gets updated to *1* by the producer, with the effect that when the consumer checks the semaphore with the *semWait* operation it finds it can simply go ahead and retrieve data from the file. Since the semaphore's value was greater than *0*, there was no need for the consumer to wait.

Use of semaphores to model solutions

When semaphores are used to model solutions to actual problems, we would typically use a number of semaphores: some of these might be used as binary semaphores, others as blocking semaphores. When we discussed the producer–consumer problem earlier in the unit, we indicated that the use of a buffer as a shared resource requires both mutual exclusion and condition synchronisation. Hence both types of semaphore would be required.

The implementation of semaphores

It is important to understand that the operations *semWait* and *semSignal* should be implemented as atomic actions. If they were not atomic, the whole point of having semaphores would be defeated.

The atomicity of the operations can be achieved through one of the three methods discussed earlier:

- hardware support in the form of a TAS instruction;
- suppression of interrupts for the duration of the *semWait* and *semSignal* operations;
- a software solution (the beginnings of which we discussed in Subsection 3.4).

The correct design of the semaphore operations, making sure that they are atomic and also that a process cannot get deadlocked or starved on invocation of such an operation, involves careful system design decisions which we will not further discuss here. If we know that the *semWait* and *semSignal* operations have been implemented correctly – not only guaranteeing mutual exclusion, but also ensuring that no deadlock can occur and that the operations will terminate – then we have the building blocks for a concurrent system.

Semaphores today

Semaphores remain in common use when programming in languages, such as C, that do not intrinsically support other forms of synchronisation. They are the primitive synchronisation mechanism in many operating systems.

However, they are a comparatively low-level mechanism, notoriously difficult to program correctly. When used in programs modelling real problems, we noted that it is likely that a number of semaphores will be required and that each semaphore serves a distinct purpose. It is up to the programmer to remember to call the exit protocol on the correct semaphore after using a critical region protected by one of the semaphores. If one such method call is omitted, the program suffers from deadlock or starvation. Similarly, forgetting an entry protocol leads to violation of mutual exclusion. A further complicating issue is that the synchronisation code is typically dispersed throughout the program, rather than localised in well-defined regions.

On the whole, the trend in programming language development has been towards more structured forms of synchronisation, such as monitors (a topic we cover below). However, semaphores remain valuable as they offer the flexibility that is required in advanced concurrency requirements (the `Semaphore` class from the Java package for concurrency utilities will be discussed in *Unit 4*).

SAQ 4

(a) In what ways can semaphores be used?

(b) How can the incorrect use of semaphores lead to problems with safety and liveness?

ANSWER.....

(a) Semaphores can be used for:

- ensuring mutual exclusion from critical code;
- controlling access to single or multiple shared resources;
- synchronising cooperating processes.

(b) By forgetting to use *semWait* it is possible for a resource to become accidentally unprotected (a safety issue). By forgetting to use *semSignal* it is possible for a resource to become locked indefinitely (a liveness issue).

Exercise 3

Explain why it is important that the semaphore's operations `semWait` and `semSignal` are atomic.

Discussion .....

Imagine the scenario where two processes, *A* and *B*, use a semaphore for mutual exclusion. Process *A* reads the value of the semaphore as `1` during a `semWait` operation and, at the same time, process *B* also executes this method, reading the value `1`. Process *A* now sets the semaphore's value to `0` and enters its critical region. Process *B* does the same – it writes the value of the semaphore as `0` and enters its critical region. Now something bad has happened: the mutual exclusion that the semaphore was supposed to impose has been violated.

3.6 Monitors

The next step in the development of synchronisation mechanisms aimed to give the programmer more support by providing higher-level programming features. This was the idea behind **monitors**, which were introduced by Hoare in 1974, as data structures to encapsulate the shared data (Hoare, 1974). Here the shared data is accessible only through special monitor operations which are atomic.

Monitors not only facilitate mutual exclusion (through the atomic monitor procedures), they also give support to condition synchronisation, by enabling a process to be blocked until a particular condition holds, such as a buffer holding data or a count being non-zero.

The monitor construct

The monitor construct encapsulates the data that needs to be protected so that it can be accessed only using operations that are part of the monitor (see Figure 11).

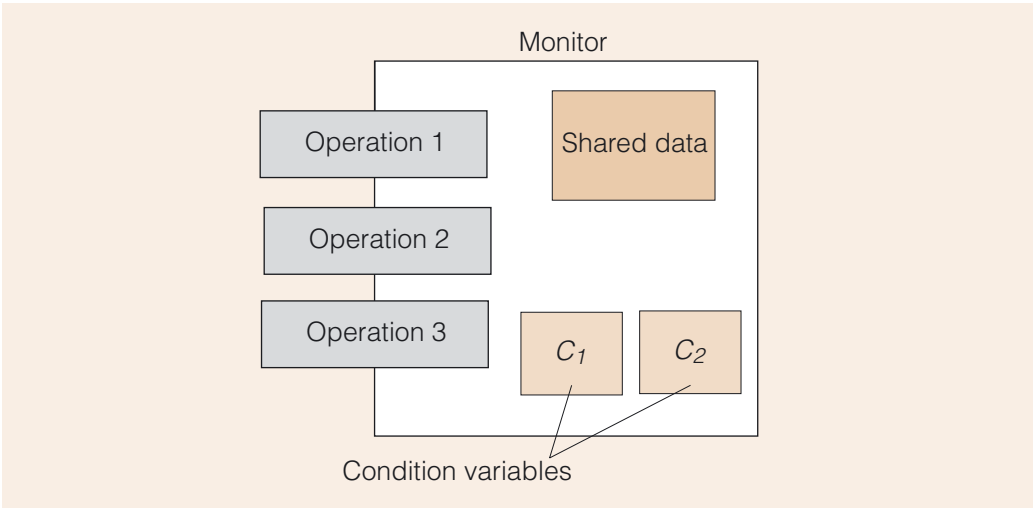


Figure 11 Monitor construct

Figure 11, as a whole, represents the monitor, which as we can see has an area of data in the middle, encapsulated within the monitor. The monitor has a well-defined interface, and only the operations defined in the interface can be used to access the monitor's data. The monitor also has a number of condition variables, which we will discuss shortly.

To understand how such a monitor data structure could be used, we compare it with the way a semaphore is used. Table 4 gives the pseudocode showing how a process can access data through a critical region, by using a semaphore as compared to a monitor.

Table 4 Using a semaphore versus using a monitor

Using a semaphore *sem*	Using a monitor *mon*

`// non-critical code`
`sem.semWait(); //entry protocol`
`// critical region`
`sem.semSignal(); //exit protocol`
`// non-critical code`
	`// non-critical code`
	`mon.operation1();`
	`// non-critical code`

When using a semaphore, a process first has to call the entry protocol before it can enter its critical region, and it must remember to call the exit protocol afterwards. However, when using a monitor, a process simply calls the appropriate monitor operation. A monitor operation is equivalent to a critical region with entry and exit protocols. This is achieved by the compiler *associating a synchronisation mechanism, such as a semaphore*, with the monitor's operations.

The squares marked C_1 and C_2 in Figure 11 represent the **condition variables**, special variables associated with the monitor that enable the monitor to handle condition synchronisation. The condition variables act in the same way as semaphores – the only operations we can use to change their value are the `semWait` and `semSignal` operations. Like a semaphore, a condition variable has a queue, so that if a condition is not satisfied, a process can be made to wait in the queue for that specific condition variable.

This means that if the monitor shown in Figure 11 had been implemented using semaphores (rather than some other concurrency mechanism) it would have required three semaphores: one semaphore to provide the mutual exclusion for the operations, and a further two semaphores for the two condition variables C_1 and C_2. Therefore, it is sometimes said that the monitor construct was built *on top of* semaphores.

The application programmer using monitors does not have to worry about how many semaphores are required – here we just mentioned that as a way of explaining how the monitor would have worked. The programmer decides what shared variables are to be encapsulated in the monitor and what operations are to be performed, and declares the condition variables required for the application, but it is the monitor implementation that manages all the low-level queuing of processes for these conditions.

Monitors as higher-level constructs

We will not discuss how to write programs using these monitor data structures further, because monitors as defined by Hoare have moved on considerably since they were first introduced. However, we did want to include them in our discussion of concurrency mechanisms, because they have developed into the major concurrency model in use at the time of writing – the Java concurrency mechanism. This mechanism will be discussed in the next section – here we want to draw your attention to the monitor as a high-level construct (and the advantages this offers).

- When a semaphore is used to guard a critical region, there is *no direct relationship* between the semaphore and the data being protected. This is part of the reason why semaphores may be dispersed around the code, and why it is easy to forget to call `semWait` or `semSignal`, in which case the result will be, respectively, to violate mutual exclusion or to lock the resource permanently.

- In contrast, neither of these bad things can happen with a monitor. A monitor is tied directly to the data (it encapsulates the data) and, because the monitor operations are atomic actions, it is impossible to write code that can access the data without calling the entry protocol. The exit protocol is called automatically when the monitor operation is completed.
- A monitor has a built-in mechanism for condition synchronisation in the form of condition variables. A process wanting to access the data protected by the monitor can check the condition variable before proceeding. If the condition is not satisfied, the process has to wait until it is notified of a change in the condition. When a process is waiting for condition synchronisation, the monitor implementation takes care of the mutual exclusion issue, and allows another process to gain access to the monitor.

The monitor was a very important development in the support given to developers dealing with complex concurrency problems.

Concurrency problems in the Therac-25

The Therac-25 was a radiation therapy machine known to have caused the deaths of a number of patients by giving massive overdoses due to software errors (these accidents happened during the 1980s). The failure of the Therac-25 software reminds us of the importance of making sure that concurrency problems are addressed properly, and of the importance of higher-level constructs that can safeguard the programmer against possible pitfalls.

The Therac-25 machine was capable of delivering two types of radiation therapy: *Electron Beam therapy* and *Megavolt X-Ray therapy*.

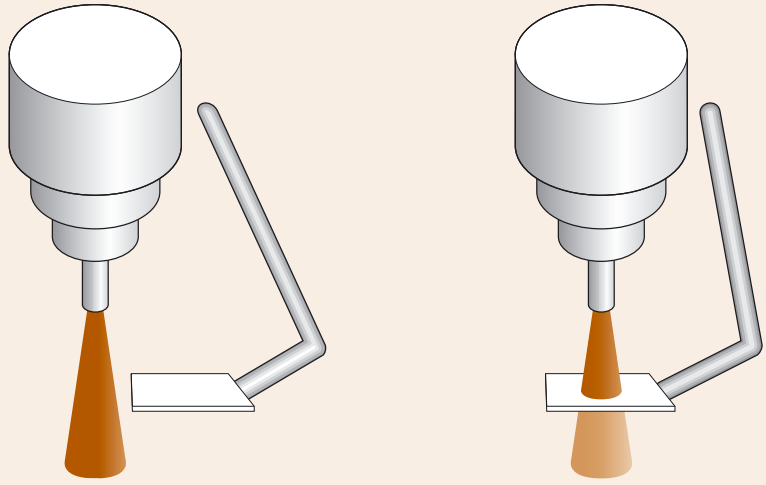


Figure 12 The two modes of operation of the Therac-25

When operating in Electron mode, the beam was emitted directly from the machine; whereas for the X-Ray mode a Tungsten shield was put in place. The accidents occurred when the high-power electron beam was activated for X-Ray therapy, without the shield having been rotated into place. The machine's software did not detect that this had occurred, and therefore did not determine that the patient was receiving a potentially lethal dose of radiation, or prevent this from occurring. This resulted in the patient being exposed to unshielded, high-energy radiation, with no indication to the operator that this was happening.

On investigation, a number of problems were noted, including the following:

1 The software to control the equipment did not properly synchronise with the software that controlled the operator interface. If the operator made a change halfway through the set-up procedure (for example, they pressed a key to change from one mode to another mode), this was not picked up by the software.

This was an example of a race condition – the exact point at which such a change was requested was crucial: under some circumstances the keyboard input was responded to, but under other circumstances it would be ignored. This was evidently missed during testing. During testing the operators were unfamiliar with the equipment and worked relatively slowly. It was only after testing, when the operators gained experience, that they worked fast enough for the problem to occur.

2 The software set a flag variable by incrementing it. Occasionally there was an arithmetic overflow in which case the software bypassed safety checks.

3 The software was written in assembly language – such low-level code is far harder to debug than most high-level languages and the developer is provided with little support for their debugging activities.

Sources: Leveson and Turner (1993) and Wikipedia (2007)

The Therac-25 machine.

4 The Java concurrency mechanism

The language mechanism built into Java to deal with concurrency is modelled on monitors. The fact that Java is an object-oriented language means that classes defined in Java can provide the same encapsulation that monitors offer. The idea that data can only be accessed through a set of well-defined operations, i.e. methods, lies at the heart of object-oriented languages.

However, Java methods are not automatically 'atomic'. It is possible that several threads could be executing the same method on shared data and we cannot simply assume that a thread invoking a method will be guaranteed to be able to complete that method without being interrupted by the scheduler.

In order to enable atomicity, all Java objects have a lock, also sometimes referred to as a monitor, or a monitor lock. The lock is acquired through the `Object` class, from which ultimately each Java object inherits. One way of looking at it is to say that the lock is attached to the object, and not to the code, and exists automatically without the programmer having to do anything.

Most of the discussion in this section is about the safety aspect of Java programming. This is often referred to as **thread safety**. A class can be said to be thread safe if it can be executed concurrently by several threads in a safe manner, i.e. without the shared data getting into an inconsistent state.

We will discuss how the Java concurrency mechanism works and how to write Java programs using this mechanism, and will use an example relating to car parks for illustration. We will then look in detail at the `Thread` states, and complete the discussion we started in *Unit 2* by filling in the gaps on how the states are affected by the concurrency features.

The section concludes with some warning notes about the problem of deadlock.

4.1 The `synchronized` keyword

If Java threads share data, we must be able to control access to that data. To this end, the Java language includes the keyword `synchronized`, which makes it possible for a method or section of code to be executed atomically. To illustrate how this keyword may be used effectively, consider the following car park problem (Magee and Kramer, 1999).

Suppose we have a class `CarParkControl` that has an instance variable `spaces` to keep track of the number of spaces available in the car park. The `CarParkControl` class has two methods, `depart` and `arrive`, where the method to model the arrival of a car in the car park is as follows:

Note that we have borrowed some features of the solution Magee and Kramer propose for this problem, but we have slightly changed the implementation.

```
public void arrive()
{
  spaces.decreaseByOne();
  System.out.print("free spaces = " + spaces.getCount());
}
```

The method `arrive` decreases the value of the counter `spaces`, and prints out the current number of free spaces. The method `depart` does the opposite and increases

the value of spaces. In this example, the spaces variable is a shared resource, and any methods that access or change this data should be executed under mutual exclusion.

However, the method arrive, as with all ordinary Java methods, can be executed by a number of threads at the same time. We have seen that if several threads access shared data concurrently, this can be problematic. Therefore, we use the Java synchronized modifier in the header of a method as follows:

```
public synchronized void arrive()
{
  spaces.decreaseByOne();
  System.out.print("free spaces = " + spaces.getCount());
}
```

If a thread executing a synchronised method has got hold of the lock for the object the method has been invoked on (in this case the CarParkControl object), it cannot be interrupted by another thread executing a synchronised method for the same object. If a thread comes to the point where it wants to execute a synchronised method, but the lock is currently held by another thread, it cannot proceed and becomes blocked.

We can also use the synchronized keyword for a code block within a method. This would mean that only part of the method is treated atomically. For a block of code the synchronized keyword comes before opening and closing brackets as follows:

```
synchronized(ObjectReference) { /* Block body */ }
```

The value in parentheses indicates the object for which the thread needs to obtain the lock. The curly brackets that follow this, demarcate the code that should be executed under mutual exclusion for that object. For the case of arrive we could have written:

```
public void arrive()
{
  synchronized(spaces)
  {
    spaces.decreaseByOne();
    System.out.print("free spaces = " + spaces.getCount());
  }
}
```

In this example it is probably clearer to use synchronized in the method header as this helps to 'announce' the synchronisation implications to users of the class. In other cases, where large parts of the method could safely be executed concurrently, it may make sense to protect only the block(s) of code where mutual exclusion is essential, thus increasing efficiency.

Note that a thread executing a synchronised method holds the lock for the object concerned, but this does not stop other threads executing normal, non-synchronised methods on the same object. This is because execution of normal methods proceeds without the need for a thread to acquire a lock on the object. These other threads, executing non-synchronised methods, have full and uncontrolled access to the data, and the scheduler may very well interleave their operations with the synchronised code. If this occurred the data could be put into an inconsistent state. Hence the safest, but not always the most concurrent, strategy is to design **fully synchronised objects**, also known as **atomic objects**. In such an object *all* methods are synchronised, and there is no violation of the encapsulation principle, by which we mean that there is no public access to the data fields of the object apart from through such synchronised methods.

When dealing with inheritance and subclassing, you should be aware that the synchronisation aspect is not automatically inherited when a method is overridden. If a method is declared synchronized in the parent class, but is overridden by a subclass,

then in the subclass it should be declared `synchronized` again. Otherwise it will be executed in the normal way, without the acquisition of a lock.

Furthermore, it is not possible for a method to be declared `synchronized` in an interface, so interfaces *cannot* be used to enforce atomicity of methods in an implementing class.

We must also bear in mind that `synchronized` applies to an object of a class, not to a class as a whole. So in this example the methods become atomic for each instance of `CarParkControl`, not for all the possible instances of `CarParkControl` taken together.

4.2 Condition synchronisation in Java

Condition synchronisation is implemented in Java through use of the **wait–notify mechanism**, using the methods `wait` and `notify` or `notifyAll`. We use this mechanism when there are certain conditions that must hold true before some code should proceed. Taking the example of the car park scenario, cars can be parked only if there is space for them, and similarly, cars can depart from the car park only if there were cars parked there in the first place.

In Java, a thread invoking the `wait` method affects the state of this thread. That is, the thread will give up its hold of the lock for that object, and enter the `WAITING` state. In order for it to exit from the `WAITING` state, it must be notified. The notification will be done by another thread, which has hopefully now made the condition true, so that the waiting thread can proceed.

We noted earlier that all Java objects have a lock, and that for all Java objects we can define synchronised methods, by using the `synchronized` keyword in the method header. Similarly, `wait` and `notify` are methods that can be used on any Java object, as they are methods of the `Object` class. Given that all objects ultimately inherit from the `Object` class, this means that the wait–notify mechanism is available for all objects. The only restriction is that the wait–notify mechanism can be used *only* within synchronised methods.

If condition synchronisation were to be added to the `arrive` and `depart` methods of the `CarParkControl` class, the `CarParkControl` class could be implemented as follows.

```
public class CarParkControl
{
  protected CarParkCounter spaces;
  protected int capacity;

  CarParkControl(int n)
  {
    capacity = n;
    spaces = new CarParkCounter(capacity);
  }
```

We have used the notation `this.wait()`, rather than just `wait()` to emphasise that the `wait` is invoked on an object, in this case an object from the `CarParkControl` class. You will also come across the use of `wait` just by itself, which is, of course, equivalent to `this.wait()`.

```
    public synchronized void arrive() throws InterruptedException
    {
      while (spaces.getCount() == 0)
      {
        this.wait();
      }
      spaces.decreaseByOne();
      System.out.println("Arrived - free spaces = " + spaces.getCount());
      this.notifyAll();
    }

    synchronized public void depart() throws InterruptedException
    {
      while (spaces.getCount() == capacity)
      {
        this.wait();
      }
      spaces.increaseByOne();
      System.out.println("Departed - free spaces = " + spaces.getCount());
      this.notifyAll();
    }
  }
```

In the `arrive` method the variable `spaces` is checked in the `while` loop condition, and if this has a non-zero value, then the `arrive` method can proceed; otherwise it has to wait. The `depart` method checks whether the car park is currently empty, in which case the `depart` thread has to wait.

At the end of both methods, the variable `spaces` is updated with a new value, the result is printed and other threads are notified. The object's lock is released when the `synchronized` method completes. Note that in this example we have used `notifyAll` rather than `notify`. We will discuss the difference between these two methods in a moment.

The `wait` method can throw an `InterruptedException` if the calling thread is interrupted while waiting. Hence the `throws` clause was added to the signature of the `arrive` and `depart` methods to indicate to the users of the methods that this exception might be thrown. This facility is useful, because if a thread is waiting for a condition, there is always a possibility that this condition may never become `true`, and that the thread could not come out of the `WAITING` state. Being able to interrupt such a thread is a way of handling this problem, but how to process such a thread is application-specific, and depends on the algorithm for handling the interruption. We will not discuss how this is done.

The methods `wait` and `notify` or `notifyAll` form a pair working together to achieve condition synchronisation. Condition synchronisation is also sometimes referred to as **guarded suspension** – meaning that a method invocation is suspended until a condition (i.e. a guard) holds. In this course we will mainly use the term *condition synchronisation*. However, should you come across terms like 'guarded methods' you should understand that this refers to methods that have a condition and that the condition may need to be waited on. Both condition synchronisation and guarded suspension refer to the notion that a condition may not hold *at this point in time*, but that it may become true at a later point. Until such time, the thread is made to wait.

Doug Lea (1999, p. 184) describes the general technique to implement condition synchronisation as follows.

- For each condition that needs to be waited on, write a `wait` loop that is guarded by a condition which causes the current thread to block if the condition is false.

- Ensure that every method causing state changes that affect the truth value of any waited-for condition notifies threads waiting for state changes, causing them to wake up and recheck their conditions.

This is what we did for the method `arrive`. We provided a `wait` loop to make the thread wait for as long as the condition does not hold. The method `arrive` changes the state of the variable `spaces`, hence we use `notifyAll` to alert potentially waiting threads.

The `wait` method shown above (without any argument) has a variant, which is the wait with a single parameter indicating the maximum length of time (in milliseconds) for which a thread is prepared to be blocked. This is called a timed-wait or a timeout.

Activity 3.2

The car park problem.

Activity 3.3

Producer–consumer problem – adding synchronisation.

4.3 Thread states, locks and the wait set

The methods `wait` and `notify/notifyAll` have the effect of changing the state of the thread. In *Unit 2* we discussed the Java thread states, and the transitions from one state to the other. We are now in a position to fill in some of the details for the diagram we studied in *Unit 2*.

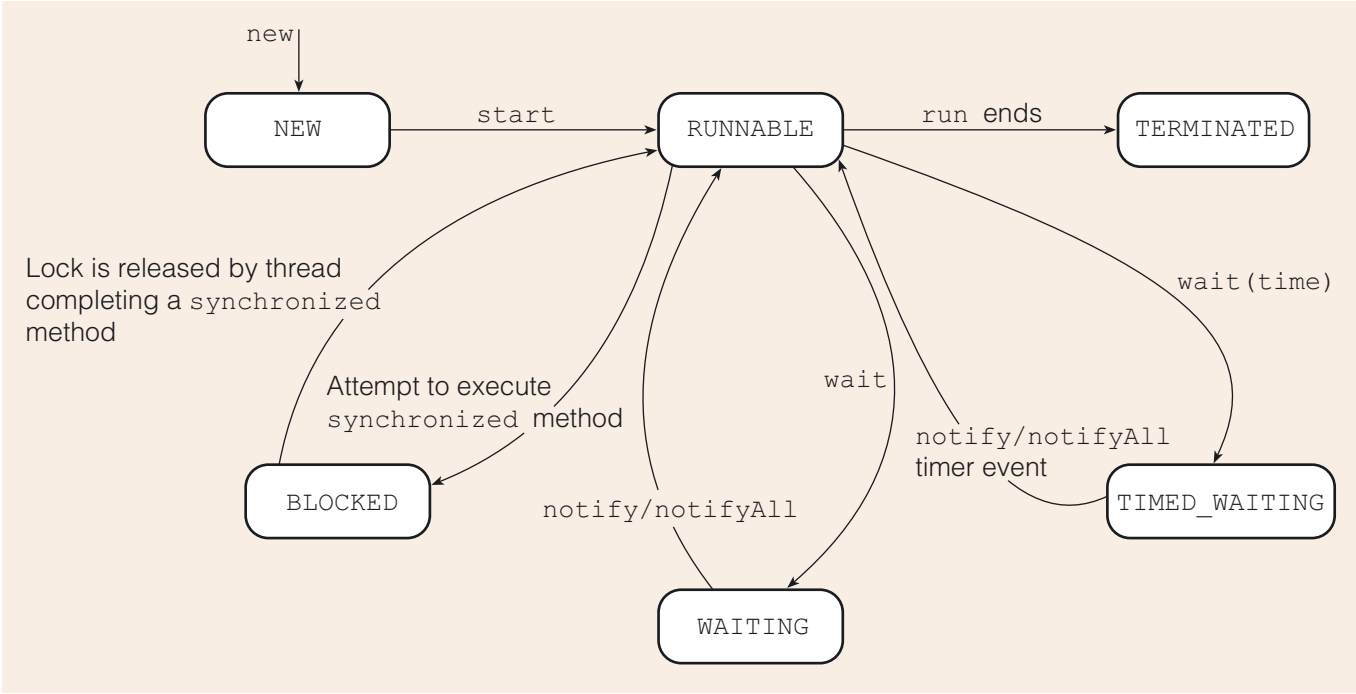


Figure 13 Java thread state transitions using `wait`, `notify` and `notifyAll`

Consider the following situation. A thread, say thread *A*, is currently executing. This means it is in the `RUNNABLE` state, and it has been picked by the scheduler to run. At some point thread *A* may come to a point where it needs to execute a `synchronized` method on some object. Before it can execute this method it must first acquire the lock associated with the object. If the lock is currently free, thread *A* can acquire it and proceed immediately. However, if the lock is already being used by some other thread (say thread *B*), thread *A* will enter the `BLOCKED` state until such time as the lock becomes free.

In due course thread *B* will give up the lock and thread *A* will have a chance to acquire it, although it will have to compete with any other threads that wish to gain possession of the lock for that object. So, it is possible that thread *A* will have to wait again. Sooner or

later, however, thread *A* will succeed in gaining the lock and can then start executing the `synchronized` code.

If the method has been declared `synchronized` only in order to prevent more than one thread at a time accessing the object (i.e. simply to enforce mutual exclusion) then thread *A* will proceed to execute all the method code enabling the method to return, at which point thread *A* will give up the lock again.

However, there may also be a need for condition synchronisation: thread *A* may have to ensure that the object the method is being executed on is in a suitable state before continuing. For example, a consumer thread will need to check that a buffer is not empty before it removes an item. If thread *A* checks the state of the object and finds that some condition is not satisfied, it will invoke the `wait` method on that object. This will cause thread *A* to enter the `WAITING` state, where it will remain until it is awakened. Threads waiting for condition synchronisation are placed in a **wait set**.

Thread *A* will also have to give up ownership of the lock, so that some other thread can gain possession of it and do some processing on the object. This is essential, since otherwise the state of the object will never change and consequently it will never become possible for thread *A* to continue.

Now imagine that a third thread (thread *C*) gets hold of the lock for the object and is able to execute another `synchronized` method which changes the state of the object. (In a producer–consumer example where *A* was the consumer, *C* might be the producer.) When *C* has finished executing the synchronised code it will invoke either `notify` or `notifyAll` on the object. These methods have the effect of awakening either one or all of the threads that are waiting on the same object.

The methods `wait`, `notify` and `notifyAll` may only be called from within synchronised code. The thread invoking them must hold the lock for the object, otherwise a runtime exception will occur.

- The `notify` method will awaken a *single* thread, chosen arbitrarily from the wait set; we cannot assume that it will be any particular thread.
- The `notifyAll` method will awaken *all* the threads in the wait set. If the wait set should happen to be empty then the notification will simply be discarded.

When the `synchronized` method completes, thread *C* will automatically surrender the lock and it becomes available to other threads again.

Note that when competing for the lock, the fact that *A* has been awakened will not afford it any special privileges over other threads waiting for the same lock.

Imagine now that thread *A* is awakened as a result of a notification from thread *C*. This will cause thread *A* to enter the `RUNNABLE` state, and at some point thread *A* will be scheduled to execute, although exactly when this happens is not predictable. Thread *A* will then compete again for the lock and when it succeeds in gaining the lock the `wait` method can return. Thread *A* will then resume execution of the synchronised code from the point immediately after the point at which `wait` was invoked.

Threads in the car park

We can illustrate the above by looking at an example of three threads using the same `CarParkControl` object: thread T_1 invokes the `depart` method, whereas thread T_2 and thread T_3 both invoke the `arrive` method. Thread T_1 is the first to start running, and we assume that at that point the car park is empty. Here is a possible scenario of what might happen.

1 Thread T_1 invokes the `depart` method and acquires the synchronisation lock.

2 Thread T_1 examines the condition and determines that the data is not in the desired state.

3 Thread T_1 invokes the `wait` method, which frees the lock.

4 The scheduler allows thread T_2 to run, and it invokes the `arrive` method, acquiring the synchronisation lock.

5 Now thread T_3 gets a chance to run, and it attempts to invoke `arrive`, but ends up in the `BLOCKED` state waiting for the lock.

6 Thread T_2 continues to run, checks the condition and finds that it can continue without having to call `wait`. It changes the data and calls the `notifyAll` method.

7 Thread T_2 finishes the `arrive` method and frees the lock.

8 Thread T_3 changes state, and becomes `RUNNABLE` again.

9 Thread T_1 receives the notification and becomes `RUNNABLE`. Threads T_1 and T_3 now both try to get hold of the lock. Assuming that thread T_1 obtains the lock, it comes out of the `wait` method, and checks the `while` condition. The thread now finds that the condition is satisfied, so it processes the data, and calls the `notifyAll` method.

10 Thread T_1 exits the `depart` method and gives up the lock.

11 Thread T_3 now has a chance to run, acquires the lock and checks the condition, which is found to be satisfied. It changes the data and sends a notification and completes the method. The lock is freed up.

The above scenario is not uncommon when multiple threads are executing. It is particularly important to note that a thread woken up by a notification cannot assume that the condition it was waiting for will now hold. This is because the notification is sent to the object's wait set, and not to particular conditions within that set. It is quite possible that the condition for this thread has not changed at all, and it needs to continue to wait.

Of course, it is also possible that the condition *had* changed, but that in the meantime some other thread has executed and altered the state of the data again. This is why it is important that a `wait` call is placed within a `while` loop that tests whether the condition is true rather than allowing the thread to continue with the risk of ignoring the condition. After all, the wait–notify mechanism does not specify the condition itself, but only provides the mechanism with which threads can communicate. The details of the condition are held in the specification of the `while` loop.

This last point brings up the subtle difference between a monitor's condition variable and Java's mechanism of associating one wait set with each object. This is sometimes referred to as Java having a **single anonymous condition variable**. By this, we mean that *one* wait set collects all the threads waiting for any number of conditions associated with an object. In the car park problem, the wait set could contain some threads that are waiting because the car park is empty, and others that are waiting because the car park is full. In a monitor, multiple conditions would be modelled using multiple condition variables, and hence a more refined notification system could be utilised. That is, in a monitor a thread is notified only under very certain circumstances, where in Java *all threads* conditionally waiting on *any number of conditions* get notified.

4.4 Locks and deadlock

When using the Java concurrency mechanism, we need to be aware of some subtle locking issues. On the positive side, the Java concurrency mechanism has removed some of the potential deadlock problems that we saw with previous constructs. However, the developer still needs to watch out for some circumstances that can give rise to deadlock.

`notify` versus `notifyAll`

There is a difference between `notify` and `notifyAll`. We saw that if `notify` is called, exactly *one thread* is notified, whereas with `notifyAll` *all threads* waiting on the object get notified. When all threads get notified this does not mean that they all start to run in parallel, it simply means that they all receive the notification, and in turn may try to obtain the lock. This may seem counter-intuitive, and you may be tempted to think that in general it is more sensible and safer to notify just one thread. However, this is not so.

Activity 3.4
Thread states and the use of `notify` and `notifyAll`.

In using `notify`, one arbitrarily selected thread (the selection taking place not by the programmer, but by the JVM thread scheduler) gets woken up, will grab the lock, and test the condition in the `while` loop. If the condition is not satisfied, the thread will call `wait` and then return to its `WAITING` state. At that point the notification has effectively been lost and the system ends up in deadlock because there is now no thread that can be called upon to execute, or notify others, or generally alter the state of the data and hence the conditions.

Generally speaking, it is safer to use `notifyAll`, and to write programs in such a way that whichever thread gets selected by the JVM, the program will work. Only in certain circumstances may the use of `notify` be useful and even improve the program's performance (for example when all threads are known to be waiting for the same condition).

Releasing the lock

When discussing semaphores, we noted that one of the potential risks is that the programmer 'forgets' to call the *`semSignal`* operation, hence bringing the system into deadlock. Fortunately, in Java the acquiring and releasing of locks is governed purely by use of the `synchronized` keyword. This means that it is not possible to 'forget' to release a lock, as this happens automatically when the invoked method stops executing. This happens even in the case that the exit of the method was due to an exception.

Re-entrant locking

The locking mechanism used by Java is known as a **re-entrant locking**. By this we mean that if a thread holds the lock of an object, and it invokes another synchronised method *for the same object*, then it is allowed to continue. This means that under these circumstances no deadlock will arise.

The way this is achieved is that the JVM associates a counter with each lock. If a thread holding a lock acquires the lock again, the counter is incremented, and it is decremented on release. If a thread, while inside the execution of a synchronised method, calls another synchronised method on the same object, the lock is released only after the thread has returned from both calls. The programmer cannot gain access to the locks or to these counters.

Multi-party operations

Multi-party operations are operations that involve access to several objects. If such multi-party operations are synchronised, they require several locks: one for each object in the operation. The programmer should be aware of the potential of deadlock when using such methods. This is shown with the example of a class `Square` – each instance of this class has a variable `colour`, a getter and a setter method for `colour`, and a method `swapColour` which allows two squares to swap their colours. All three methods are synchronised:

Based on an example by Doug Lea (1999).

```
class Square                // warning: do not use this class
{
  private String colour;

  public Square(String col)
  {
    colour = col;
  }
```

```
    public synchronized String getColour()
    {
      return colour;
    }

    private synchronized void setColour(String c)
    {
      colour = c;
    }

    public synchronized void swapColour(Square other)
    {
      String c1 = this.getColour();
      String c2 = other.getColour();
      this.setColour(c2);
      other.setColour(c1);
    }
  }
```

The method `swapColour` takes as its argument another `Square` object reference. It works by setting the colour value of the invoking `Square` object to that of the colour of the argument `Square` object, and vice versa. Hence, when executing the method `swapColour` the thread has to own the lock of the invoking object, as well as the lock for the object to swap colour with.

It is possible for two different threads, T_1 and T_2, both using the same two objects, `squareA` and `squareB`, to become deadlocked when the following executions take place:

thread T_1 executes `squareA.swapColour(squareB)`

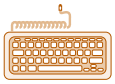
and

thread T_2 executes `squareB.swapColour(squareA)`.

Table 5 shows a sequence of events that could lead to deadlock.

Table 5 Sequence of events leading to deadlock in a multi-party operation

Thread T_1	Thread T_2
Acquire the lock for `squareA` on entering `SquareA.swapValue(squareB)`.	
Succeed with `c1 = this.getColour();` because the lock for `squareA` is held already.	
	Acquire the lock for `squareB` on entering `squareB.swapColour(squareA)`.
Block on the invocation of `other.getColour()` because it requires the lock for `squareB`.	
	Succeed with `c1 = this.getColour();` because the lock for `squareB` is held already.
	Block on the invocation of `other.getColour()` because it requires the lock for `squareA`.



Activity 3.5

Deadlock in multi-party operations.

At that point the two threads are held in deadlock, as they are both blocked, and there is no prospect of either obtaining the lock it is waiting for. We will return to the subject of deadlock in Section 6.

SAQ 5

(a) How can we achieve mutual exclusion in Java?

(b) What is the wait–notify mechanism designed to assist with?

(c) What is the distinction between the following three thread states: `BLOCKED`, `WAITING` and `TIMED_WAITING`?

(d) What is the difference between `notify` and `notifyAll`, and which of these two methods is considered safer to use?

ANSWER.....

(a) In Java, mutual exclusion can be achieved by using synchronised methods, using the `synchronized` keyword. Fully synchronised objects are objects for which all the methods are synchronised, and that is the safest, but possibly not the most efficient, strategy. For some objects, only certain operations or parts of operations need to be carried out under mutual exclusion, so the objects do not need to be fully synchronised.

(b) The wait–notify mechanism provides a way to implement condition synchronisation in synchronised methods – a thread can wait on a condition if the condition is not satisfied, and threads may notify other threads if they have altered the state of the shared data.

(c) A thread ends up in the `BLOCKED` state if it attempts to execute a synchronised method, but the lock for the object is already held by another thread. Both `WAITING` and `TIMED_WAITING` are states that a thread ends up in as a result of the `wait` method, and hence as a result of a condition not being satisfied. A thread in the `TIMED_WAITING` state can be woken up as a result of either a notification or the timeslot having expired.

(d) With `notify` only one thread gets selected to receive the notification – but we do not know which thread it will be. With `notifyAll` all threads get notified. On the whole `notifyAll` is safer to use, because with `notify`, if the wrong thread receives the notification, the program may end up deadlocked, as the notification cannot be passed on to other threads.

5 The multiple readers, single writer problem

In this section we discuss the **multiple readers, single writer** problem. This is a 'classic' and important problem in the study of mutual exclusion and condition synchronisation, and brings together many of the issues we have studied in this unit.

We will begin with the basic version of this problem, and then, by fine-tuning the problem, look at how it is possible to have different policies.

5.1 Problem description

The multiple readers, single writer problem is best understood in the context of a real example. Suppose that an application consists of a number of threads that all require access to a single file held on disk (this is the shared data). Some of the threads aim to read the contents of the file – these are the *readers*. Some of the threads want to write to the file – these are the *writers*.

If several threads access a shared resource in order to write to it, this can be problematic. We discussed this in Subsection 3.1 with a scenario of uncontrolled access to a shared resource where an update made by one process was lost because it was overwritten by another process (the lost update problem). It is therefore necessary that a shared resource is accessed only under mutual exclusion. Therefore, for writing purposes, we restrict access to a shared resource to a *single writer.*

However, since *reading* from a file does not change the file, we can allow multiple readers to access the file concurrently. If multiple threads are permitted to read at the same time, we would expect better performance.

5.2 Data to track the readers and writer

In order to model this we need to keep track of two pieces of data: how many threads are currently reading, and whether there is a writer thread currently active. Hence we need to have the following variables:

```
int activeReaders  // number of readers currently reading (>= 0)
int activeWriters  // number of writers currently writing (0 or 1)
```

Before a reader thread begins reading, it should test whether it is safe to do so, by checking the value of the `activeWriters` variable. Similarly, a writer thread should check the values of both variables, since it is allowed to write only if no threads are actively using the shared resource in either role.

After a writer has finished writing, and after a reader has finished reading, the states of the variables `activeWriters` and `activeReaders` should be updated, to reflect the current situation.

To model the multiple readers, single writer problem, we can therefore apply the idea of entry protocols and exit protocols for the reading and writing tasks. Note that it will be necessary to make sure that when checking the value of, say, the variable `activeReaders`, that this happens under mutual exclusion, to avoid race conditions.

5.3 Readers and writers: abstract class

There are different ways in which we can implement the multiple readers, single writer problem in Java, and here we discuss the approach in which we define an abstract class `ReadWrite`. Recall that objects cannot be created directly from abstract classes, but only from the subclasses where the abstract methods are implemented. Typically, some methods are deferred in an abstract class, to be fully implemented in the subclass.

The Java code for class `ReadWrite` is as follows:

```
public abstract class ReadWrite
{
  protected int activeReaders = 0; // number of readers currently reading
  protected int activeWriters = 0; // number of writers currently writing

  protected abstract void doRead();          // implemented in a subclass
  protected abstract void doWrite(Object o); // implemented in a subclass

  public void read() throws InterruptedException
  {
    beforeReading();              // invoke entry protocol
    try
    {
      doRead();                   // critical region
    }
    finally
    {
      afterReading();             // invoke exit protocol
    }
  }

  public void write(Object o) throws InterruptedException
  {
    beforeWriting();              // invoke entry protocol
    try
    {
      doWrite(o);                 // critical region
    }
    finally
    {
      afterWriting();             // invoke exit protocol
    }
  }

  private boolean allowReading()
  {
    return activeWriters == 0;
  }

  private boolean allowWriting()
  {
    return activeReaders == 0 && activeWriters == 0;
  }
```

```
  private synchronized void beforeReading()
          throws InterruptedException          // entry protocol for read
  {
    while (! allowReading())
    {
      this.wait();
    }
    ++activeReaders;
  }

  private synchronized void afterReading()     // exit protocol for read
  {
    --activeReaders;
    this.notifyAll();
  }

  private synchronized void beforeWriting()
          throws InterruptedException          // entry protocol for write
  {
    while (! allowWriting())
    {
      this.wait();
    }
    ++activeWriters;
  }

  private synchronized void afterWriting()     // exit protocol for write
  {
    --activeWriters;
    this.notifyAll();
  }
}
```

The abstract class `ReadWrite` has two public methods: `read` and `write`. It is these methods that will be used by threads to request a read or write operation. The `read` method is structured as having an entry protocol (here `beforeReading`), followed by the actual critical region (here method `doRead`) and finally the exit protocol (here `afterReading`). The `write` method is structured along similar lines.

The methods `doRead` and `doWrite` are abstract, and will be implemented in the subclass that will inherit from class `ReadWrite`. The advantage of organising matters this way is that the entry and exit protocols are *enforced* by the abstract superclass.

Furthermore, the abstract class can be used by readers and writers using any type of data, whether this is data held in a file, in a linked list, or on a web page. The important logic of the multiple reader, single writer model has been captured in this abstract class, making this a reusable solution for many different readers and writers.

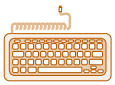
The signatures of the methods `read` and `write` indicate that these methods can throw an exception. This is ultimately because the entry protocol (e.g. `beforeReading`) has a call to the `wait` method, which we know can throw an exception.

You will also notice that the `read` and `write` methods use a `try/finally` clause. Such clauses can be used when a section of code can have multiple exit points: for example, a process may have completed normally, may have come out of a section of code through a `break` statement, or may have come to an end through an exception. In some scenarios it is important to be able to say ‘no matter how this particular bit of code was

exited afterwards, this other bit of code *must* be executed'. The `try`/`finally` clause is structured as follows:

```
try
{
  // block of code with multiple exit points
}
finally
{
  // block of code that is always executed when the try block is exited,
  // no matter how the try block is exited
}
```

In our scenario, we do not know anything about the implementation of `doRead` or `doWrite`. As a precaution we therefore put the `doRead` and `doWrite` calls in `try` blocks, and in the `finally` part we execute the exit protocol, `afterReading` or `afterWriting`, thus ensuring that these protocols will always be executed, no matter how the `doRead` or `doWrite` method exited.



Activity 3.6

The multiple readers, single writer problem.

It is also worth spending some time studying the exact working of the method `beforeReading`. It has a `while` condition which tests whether the method `allowReading` returns a `true` or `false` value. As long as this is `false`, the method is waiting. When the condition changes, and the `while` now evaluates to `true`, the statement to increment the `activeReaders` variable gets executed. This is because at that point, the thread is at the end of the entry protocol, and will actually continue into the processing of the critical region, where it will be the case that there is now one more active reader.

5.4 The writers-preferred policy

The abstract class for the multiple readers, single writer problem we have just seen is the 'basic' solution model for this problem. With some adjustments, we can add more subtle policies regarding which type of process should receive preferential treatment. There are a number of different policies that can be adopted – which policy is the best depends on the nature of the application. Here we list five possible policies.

1 The basic multiple readers, single writer policy – the shared resource can be accessed by a number of readers concurrently, or by one writer exclusively.

2 The writers-preferred policy – the shared resource is given to a waiting writer (if there is one) and only if there is no writer waiting will it be given to readers. All waiting readers will be allowed to access the structure at the same time.

3 The readers-preferred policy – the shared resource is given to all waiting readers (if there are any) and is given to a writer only if there is no reader waiting.

4 The alternating readers-writers policy – the shared resource is given to a waiting writer when readers have finished with it and to readers when a writer has finished with it.

5 The take-a-number policy – the shared resource is given to the threads in order of arrival. It requires the threads to check to see whether the number they have taken is now being served.

We will now explore in more detail how to implement different policies, using as an example the writers-preferred policy. This policy is used in applications where it is preferable that readers read the most up-to-date version of the data resource.

The idea behind the writers-preferred policy is that if the shared resource, say a file, is being read by a number of readers, and if, in a such a scenario, a *writer* requests to write

to the file, no further readers will be allowed to access the file until after the writer has completed its task. The current readers are allowed to complete their tasks, but when they have done so, the writer is given access. Thus, when several readers are reading, if there is at least one writer waiting, any new readers wishing to read must wait until all writers have finished before they can gain access.

First we will show how we can amend the class `ReadWrite` so that it is generic, and can be used by a number of different policies. We then show how a specific policy (writers-preferred) can be implemented in a subclass of this generic `ReadWrite` class.

For the writers-preferred policy we need to be able to reason about whether any writers are currently *waiting*. Hence, we need to add a variable: `waitingWriters`. However, since we want a generic class we will also introduce a variable `waitingReaders`. At various points in the code these variables get checked and/or updated, in order to implement the appropriate policy.

```
public abstract class ReadWrite      // generic version
{
  protected int activeReaders = 0;   // number of readers currently reading
  protected int activeWriters = 0;   // number of writers currently writing

  protected int waitingReaders = 0; // number of readers currently waiting
  protected int waitingWriters = 0; // number of writers currently waiting

  protected abstract void doRead();          // implemented in a subclass
  protected abstract void doWrite(Object o); // implemented in a subclass

  public void read() throws InterruptedException
  {
    beforeReading();              // invoke entry protocol
    try
    {
      doRead();                   // critical region
    }
    finally
    {
      afterReading();             // invoke exit protocol
    }
  }

  public void write(Object o) throws InterruptedException
  {
    beforeWriting();              // invoke entry protocol
    try
    {
      doWrite(o);                 // critical region
    }
    finally
    {
      afterWriting();             // invoke exit protocol
    }
  }
```

```
protected boolean allowReading()
{
  return activeWriters == 0;
}

protected boolean allowWriting()
{
  return activeReaders == 0 && activeWriters == 0;
}

private synchronized void beforeReading()
  throws InterruptedException              // entry protocol for read
{
  ++waitingReaders;
  while (! allowReading())
  {
    try
    {
      this.wait();
    }
    catch (InterruptedException ie)
    {
      --waitingReaders;                    // rollback state
      throw ie;                            // rethrow the caught exception
    }
  }
  --waitingReaders;
  ++activeReaders;
}

private synchronized void afterReading()   // exit protocol for read
{
  --activeReaders;
  this.notifyAll();
}

private synchronized void beforeWriting()
  throws InterruptedException              // entry protocol for write
{
  ++waitingWriters;
  while (! allowWriting())
  {
    try
    {
      this.wait();
    }
    catch (InterruptedException ie)
    {
      --waitingWriters;                    //rollback state
      throw ie;                            //rethrow the caught exception
    }
  }
  --waitingWriters;
  ++activeWriters;
}
```

```
  private synchronized void afterWriting()     // exit protocol for write
  {
    --activeWriters;
    this.notifyAll();
  }
}
```

The above version of the `ReadWrite` class differs from the basic version in the two entry protocols. For example, in the entry protocol `beforeWriting` you will note that the variable `waitingWriters` is incremented before the `while` loop, and decremented after the `while` loop. It is important to record the number of processes waiting to write, because this information is used to determine whether a reading process is allowed to read or not. A process that invokes the `beforeWriting` method must be a writer process that wants to write, hence it needs to be added to the count of waiting writer processes. As soon as the `allowWriting` method returns `true`, the process can go ahead and actually start writing, which is why the count of `waitingWriters` should be reduced again.

The exception handling in the two entry protocol methods is now more sophisticated. In `beforeWriting` we see that variable `waitingWriters` is decremented. This is necessary because the variable `waitingWriters` had been incremented at the beginning of the method. If, for some reason, the `wait` method is interrupted and exits with an exception, the action at the beginning of the method `beforeWriting` would need to be undone. The exception then gets rethrown, which has the effect of propagating the exception to the level where a thread class invokes the `read` method. This is sometimes referred to as **rolling back state**, because all the variables get returned to the state they were in before the exception occurred.

With the new version of the class `ReadWrite` it is now very simple to fine-tune some of the policies. The writers-preferred policy we discussed earlier can now be coded as a subclass from the abstract class, as follows:

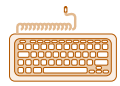
```
public abstract class ReadWrite_WritersPreferred extends ReadWrite
{
  protected boolean allowReading()
  {
    return waitingWriters == 0 && activeWriters == 0;
  }
}
```

SAQ 6

In Subsection 2.3 we discussed fairness as a liveness issue. Do you think the algorithm for the writers-preferred policy is fair?

ANSWER.....

On the face of it, it seems to make sense to give writer processes priority over readers, because the shared resource should always be in the most up-to-date state. This is presumably also in the interest of the reader processes. However, it is possible, in the given scheme, that reader processes never get a chance to run, because writer processes always get priority. This would be unfair not only to the readers, but also it would be unfair to the writers, since they would be wasting their efforts in writing things that never get read!



Activity 3.7

Multiple readers, single writer – alternative policies.

6 Deadlock

In the previous sections the problem of deadlock came up in a number of different contexts. In this section we look at a more formal definition of deadlock and discuss general strategies to deal with the problem.

6.1 The four conditions for deadlock

Four conditions have been identified as being necessary for deadlock to occur (Coffman *et al.*, 1971).

1 The processes involved share resources which they use under mutual exclusion, and a request to use a resource has been refused.

2 Processes are allowed to hold on to resources they have already acquired, while requesting other resources (a situation termed 'hold while waiting').

3 Once a process has acquired a resource, this resource cannot be taken from it forcibly (that is, there is no pre-emption).

4 A cycle of processes exists such that each process holds a resource which its successor in the cycle is waiting for.

The first three conditions can be seen as policies, whereas the last condition is a description of a particular live situation that has arisen. It is only when *all four* of these conditions hold that we have deadlock.

In order to understand the four conditions better, let us further examine the `Square` program in Subsection 4.4, which demonstrated how deadlock can occur when the same synchronised method is invoked on multiple objects (multi-party operation). In particular, we showed a sequence of events for a scenario of two threads executing methods to swap the colours of two `Square` objects:

thread T_1 executes `squareA.swapColour(squareB)`

and

thread T_2 executes `squareB.swapColour(squareA)`.

In this scenario thread T_1 has the lock on `squareA` while requesting the lock on `squareB`, but the lock on `squareB` is held by thread T_2 which is requesting the lock on `squareA`. Both threads are now in the `BLOCKED` state and this is when deadlock ensued. We can now consider each of the four conditions for deadlock with respect to this scenario.

Mutual exclusion

In this example the first condition for deadlock holds – the two threads both want to use the resources `squareA` and `squareB`, but these objects can be accessed by only one thread at a time since they are fully synchronised objects.

Hold while waiting

The second condition also holds. The thread T_1 holds the lock for object `squareA`, which enables it to execute the synchronised method `swapColour`. However, while holding this lock it requests the lock on `squareB`, which it cannot obtain – thus T_1 is moved into the `BLOCKED` state for `squareB`. 'Hold while waiting' is true, in that T_1 is allowed to hold on, indefinitely, to the lock on `squareA` that it has already acquired.

No pre-emption

The third condition results from the implementation of `synchronized`. In Java, if a thread executes a synchronised method, then it has the lock, and gives it up only when the synchronised method comes to an end. In this example thread T_1 has the lock on the `squareA` object, and it will not give it up until it comes to the end of the `swapColour` method. Thus the third condition holds.

Cycle of processes

The final condition is also true. The thread T_2 is waiting to obtain the lock for `squareA`, in order to complete its execution of the `swapColour` method which could release thread T_1 from its blocked state. However, it is not allowed to do so because thread T_1 is holding the `squareA` lock at this point in time. Similarly, T_1 cannot obtain the lock for `squareB`, that would enable it to complete its execution of `swapColour` and release thread T_1 from its blocked state.

6.2 Strategies to deal with deadlock

There are several techniques that can be used to deal with deadlock:

- Deadlock prevention. In the previous subsection we indicated that deadlock exists if, *and only if*, all four conditions hold. Hence, if one of these conditions does not hold, there can be no deadlock. The first three conditions are really policy statements, and it is possible to make an appropriate choice for one of these policies to be changed. In the next subsection we give examples of how we could do this.
- Deadlock detection. A number of algorithms exist which enable the detection of deadlock. In such algorithms a data structure, such as a table, is used to record all the objects in the system. The requests for locks on objects and the allocations are all noted, and through analysis of this data it is possible to detect whether or not deadlock exists. If deadlock is detected, the processes involved in the deadlock can be aborted. We will not go into the details of such algorithms any further.
- Deadlock analysis. An alternative strategy is to build systems for which we can prove that no deadlock can exist. This can be achieved through the use of certain formal approaches. This goes beyond the scope of this course, but the interested reader may wish to consult the approach outlined by Magee and Kramer (1999).

6.3 Deadlock prevention

Here we will briefly explore the idea of deadlock prevention. This approach involves implementing a system such that deadlock is not possible by arranging for one of the four deadlock conditions to not hold.

1 To relax the first condition is to say that no objects can be held under mutual exclusion. Throughout this unit we have learnt that mutual exclusion is often necessary to ensure that resources remain in a valid state. Even though there may be moments where *shareable locks*, rather than *exclusive locks* could be used, it is rarely a viable option to deny mutual exclusion. We discuss these two types of lock in *Unit 6*.

2 The second condition, about holding a resource while waiting, could be modified as follows. If a process were to request all the resources it needs for a particular task at the start of that task, rather than obtaining one resource and then attempting to get the next, then deadlock could not arise. However, this approach, sometimes

referred to as an 'all-or-nothing' approach, has some problems in practice. In some applications the exact resources required are not known at the beginning of a task, as they are dynamically identified during the processing. Furthermore, it can cause a large performance reduction, particularly if a number of scarce objects are needed.

3 The no pre-emption condition could be changed if a strategy based on deadlock detection is adopted. That is, once it has been detected that a process is in deadlock, it could be forced to give up its resources, and thus effectively be pre-empted.

4 The final condition regards the existence of a cycle of processes – if we are able to impose certain types of ordering, either on the objects or on the processes, then there would be no circularity.

Ordering the objects so that, for example, a request for object *A* would always occur before a request for object *B*, would be less wasteful than the requirement that all objects have to be requested. In some applications this type of ordering might be appropriate, but it is not generally applicable.

Another approach would be to number the processes and introduce a policy whereby, say, a process with a lower number had priority over a process with a higher number. Then, in the case of two processes requesting the same resource, the one with the lower number would have precedence. In *Unit 6* we discuss time-stamp ordering as a method of ordering processes that is free of deadlock.

SAQ 7

(a) What are the four conditions for deadlock?

(b) Generally, how can deadlock be prevented?

ANSWER.....

(a) The four conditions for deadlock are: mutual exclusion, hold while waiting, no pre-emption and a cycle of processes.

(b) By arranging for one of the four conditions not to be true,.

Exercise 4

Suppose that the method `swapColour` for class `Square`, as discussed in Subsection 4.4, had been implemented as follows.

```
public void swapColour(Square other)
{
  if (this.hashCode() < other.hashCode())
  {
    this.doSwapColour(other);
  }
  else
  {
    other.doSwapColour(this);
  }
}
```

```
protected synchronized void doSwapColour(Square other)
{
  String c1 = this.getColour();
  String c2 = other.getColour();
  int r = 0;
  this.setColour(c2);
  other.setColour(c1);
}
```

The method `hashCode` is a method that can be used on any object in Java, and returns an `int`, based on the reference to the object. If two variables reference the same object, then invoking the inherited `hashCode` method on them will yield the same result.

Explain why the above implementation is free from deadlock.

Discussion

Deadlock can be prevented by ensuring that one of the four conditions for deadlock does not hold, and in this particular implementation for the method `swapColour` we have made it impossible for the objects to be held in a cycle. By using the `hashCode` method, we obtain a number that can be used to impose some kind of ordering on the objects in the application. (That is, if two objects attempt to swap colours with each other, then it is always the object with the lowest integer value that gets locked first.) By imposing an ordering on the objects, the possibility of a cycle is removed.

7 Summary

In this unit you have studied process interaction, in particular the communication between processes that takes place when they share resources. Processes may need to synchronise with other processes because they need a particular condition to be satisfied in order to be able to make progress (condition synchronisation), or it may be deemed necessary for a process to gain exclusive use of a service or resource until it is fully completed (mutual exclusion).

We discussed two important properties of concurrent programs, safety (the concept that no bad things should happen) and liveness (the concept that something good eventually happens). The *lost update problem* and the problem of *race conditions* are examples of safety problems that may compromise the proper execution of a concurrent program.

Over the years a number of mechanisms have been developed to avoid such problems, and over the course of this unit we looked at some of these approaches. We started this discussion by considering the issue of mutual exclusion protocols, beginning with the concept of atomic operations, and the classification of such operations as being fine-grained (machine-level instructions) or coarse-grained.

We then moved on to the concept of a critical region of code being a section of code in which a shared resource is accessed and that, to meet mutual exclusion requirements, should be executed by only one process at a time. Here you met the idea of entry and exit protocols that can be used before and after a critical region respectively. The entry and exit protocols must themselves be atomic, and they may be usefully implemented using a hardware test-and-set instruction.

Next we discussed the development of concurrency mechanisms, from semaphores (data structures consisting of an integer value, a queue and two operations: `semWait` and `semSignal`) to monitors. Three different uses of semaphores were highlighted: binary, counting and blocking semaphores, each achieved through appropriate initialisation of the integer value, and by processes calling the semaphore's operations at the right point.

Monitors, the next evolutionary step in the development of mutual exclusion mechanisms, function at a higher level than semaphores. A monitor encapsulates the shared data such that it can be altered only by monitor operations. Only one process at a time can use a monitor's operations, ensuring that the data is accessed only under mutual exclusion.

Bringing our treatment of concurrency up to the present day, we looked at the Java concurrency mechanism, which is modelled on the monitor structure. Each Java object has a lock, and by declaring methods to be `synchronized`, the objects can be accessed only under mutual exclusion. Furthermore, condition synchronisation is supported by the Java wait–notify mechanism. We discussed the classic problem of a single writer with multiple readers, and how this could be implemented in Java, including the coding required for the policy that gives preferential treatment to writers.

The problem of deadlock was discussed as one of the main liveness problems in concurrent programs. We saw that deadlock can arise only when four specific conditions hold, and we discussed deadlock prevention as one of the ways of dealing with deadlock.

LEARNING OUTCOMES

When you have completed your study of this unit you should be able to do the following:

- explain the need to control access to shared resources when processes communicate;
- distinguish between different concurrency mechanisms: atomic operations, critical regions with entry and exit protocols, semaphores and monitors;
- evaluate the different concurrency mechanisms in terms of safety and liveness;
- explain how Java supports mutual exclusion and condition synchronisation;
- write concurrent programs in Java that are both safe and live;
- describe the multiple readers, single writer model;
- describe the conditions for deadlock and strategies to deal with deadlock.

Glossary

atomic action An indivisible action – that is, either the action happens completely, or it does not happen at all. See also **fine-grained** and **coarse-grained atomic actions**.

atomic objects Also known as fully synchronised objects, atomic objects are objects which have all their methods declared as `synchronized`. The data is fully encapsulated and there is no public access to the data fields of the object apart from through such `synchronized` methods.

binary semaphore A semaphore used to guard exclusive access to a shared resource.

blocking semaphore A semaphore used for the synchronisation of cooperating processes.

buffer An area of temporary storage to allow a reader and writer to operate at different rates.

busy-waiting A technique whereby a process repeatedly checks to see if a condition is true such as waiting for keyboard input or waiting for a lock to become available. See also **spin lock**.

circular buffer A buffer (data storage area) which is logically said to be circular – when a process gets to the end of the buffer, it can start again at the beginning.

coarse-grained atomic action An atomic action implemented by a sequence of fine-grained atomic actions.

condition synchronisation A mechanism for ensuring that a process is blocked if some condition is not fulfilled (for example, a producer process requires a buffer to be *not* full for it to be able to proceed).

condition variables Used in the operations of a monitor to provide conditional synchronisation.

consistent state A sensible state for a system as a whole (or the individual objects within the system) to be in, such that it conforms to a given specification. It usually refers to the values that are held by the variables in the system and whether these are within the range as set out by the specification.

counting semaphore A semaphore used to control access to shared resources – may be used for any number of resources.

critical region A sequence of instructions that access shared data.

deadlock A situation in which a process is prevented, for all time, from continuing because it is waiting for an event that will never happen, or for a resource that will never become free.

entry protocol The code that must be executed by a process prior to entering its critical region – it is designed to prevent the process from entering its critical region if another process is already in its associated critical region. An entry protocol together with its associated exit protocol should ensure mutual exclusion.

exit protocol The code that a process must execute immediately on completion of its critical region to ensure that other waiting processes may now enter their associated critical regions. Together, entry protocols and exit protocols are designed to ensure mutual exclusion.

fairness Fairness is concerned with guaranteeing that a process will be given the chance to proceed, regardless of how other processes behave.

fine-grained atomic action An atomic action implemented directly by an indivisible machine instruction.

fully synchronised objects See **atomic objects**.

guarded suspension A method invocation is suspended until a condition (i.e. guard) holds. Equivalent to **condition synchronisation**.

inconsistent state The state a system as a whole, or the individual objects within it, may end up in as a result of a violation of the specification for that system.

interrupt disabling When interrupts are disabled, the current executing process cannot be interrupted and will complete its execution.

livelock A situation where a process is continually executing an operation without getting nearer to a condition becoming true.

liveness This property asserts that a program must make some form of progress, and not come to a halt or run indefinitely without achieving its goal. See also **safety**.

lock A file or object is said to be locked when processes are prevented from accessing it (usually because another client/process is accessing it). In Java, each object has a lock.

lost update problem The situation arising when a result made by one process has been overwritten (and hence lost) by another process. This occurs if uncontrolled access to a shared resource has been allowed.

monitors So-called Hoare monitors are data structures that provide operations which access encapsulated shared data under mutual exclusion. The concurrency mechanism implemented by the Java language, with use of a lock and synchronised methods, is said to be based on monitors.

multi-party operation An operation that involves access to several objects. Since a synchronised multi-party operation requires several locks (one for each object in the operation), the programmer should be aware of the potential for deadlock.

multiple readers, single writer A classic problem in the study of mutual exclusion and condition synchronisation, in which only one writer is allowed to write at a time, but multiple readers are allowed to read.

mutual exclusion An approach that ensures only one process at a time can access a shared resource.

mutual exclusion protocol A set of rules (protocol) which, if obeyed by a number of processes that wish to access a shared resource, will enforce mutual exclusion.

producer–consumer model A model in which two processes work together and share data in such a way that one process assumes the role of producer (i.e. writes to the data resource) and the other assumes the role of consumer (i.e. reads from the resource).

protocol The rules that govern an interaction between different components.

race conditions Race conditions exist if the final outcome of a process is dependent on the timing or sequence of other processes (as if the processes are 'racing' each other).

re-entrant locking The locking mechanism used in Java. If a thread holds the lock for an object, and it invokes another `synchronized` method *for the same object*, it is allowed to continue. This means that under these circumstances no deadlock will arise.

rolling back state The act of returning the system's state (usually the variables) back to how it was before an event occurred (such as an exception).

safety This property asserts that nothing bad happens during the execution of a program. See also **liveness**.

semaphore A type of object that is used to protect a resource from concurrent access by multiple processes. It does this through the use of two operations *`semWait`* and *`semSignal`*. Internally, a semaphore has a data field that holds its value and a queue that is used to hold blocked processes.

shared resources A resource, which may be either data (in the form of variables or files) or hardware, that a number of processes have access to.

single anonymous condition variable Java's concurrency mechanism allows just one wait set for all the threads that have called `wait` on a given object's lock. This acts as the condition variable. This is in contrast to so-called Hoare monitors which can have multiple condition variables, and a queue for each specific condition. Therefore, we say that Java has a single condition variable, which is anonymous as it does not belong to any particular condition.

spin lock A lock where the thread simply waits in a loop ('spins'), repeatedly checking until the lock becomes available. This is also known as **busy-waiting** because the thread remains active but is not performing a useful task.

starvation Starvation occurs when a runnable process is overlooked indefinitely by the scheduler.

synchronisation The need for different processes to synchronise their operations by waiting for each other. In Java, the keyword `synchronized` is used for methods, so that only one thread can execute the method at a time, forcing the other threads to wait.

test-and-set (TAS) A special atomic operation that tests and sets a Boolean, provided by the operating system as an indivisible machine instruction.

thread safety A class is thread safe if it can be executed concurrently by multiple threads without the shared data getting into an inconsistent state.

wait set The set of threads that have called `wait` and are waiting to be notified. Each object in Java has one wait set.

wait–notify mechanism The Java mechanism to implement condition synchronisation. A thread calls `wait` and will be blocked as long as the condition it is waiting on is not satisfied. Another thread calling `notify` or `notifyAll` can release this thread from its blocked state.

References

Andrews, G.R. (1991) *Concurrent Programming, Principles and Practice*, Benjamin/Cummings.

Coffman, E.G. Jr, Elphick, M.J. and Shoshani, A. (1971) 'System deadlocks', *ACM Computing Surveys*, vol. 3, no. 2.

Dijkstra, E.W. (1965) 'Solution of a problem in concurrent programming control', *Communications of the ACM*, vol. 8, no. 9, p. 569.

Dijkstra, E.W. (1968) 'The structure of the THE operating system', *Communications of the ACM*, vol. 1, no. 5.

Hoare, C.A.R. (1974) 'Monitors: an operating system structuring concept', *Communications of the ACM*, vol. 17, no. 10, pp. 549–57; also available online at http://www.acm.org/classics/feb96 (Accessed 24 August 2007).

Lea, D. (1999) *Concurrent Programming in Java* (2nd edn), Prentice Hall PTR.

Leveson, N. and Turner, C.S. (1993) 'An investigation of the Therac-25 accidents', *IEEE Computer*, vol. 26, no. 7, pp. 18–41.

Magee, J. and Kramer, J. (1999) *Concurrency: State Models and Java Programs*, Wiley.

Poulsen, K. (2004) *Tracking the Blackout Bug* [online], SecurityFocus, http://www.securityfocus.com/news/8412 (Accessed 6 June 2007).

Wikipedia (2007) *Therac-25* [online], http://en.wikipedia.org/wiki/Therac-25 (Accessed 24 August 2007).

Index